

10 Habits tracker using gRPC

This chapter covers

- Writing a web service using Protobuf and generating the Go code of its gRPC definition
- Using the Context interface in Go
- Running the service with basic endpoints
- Testing with integration tests

As developers, we spend most of our work day in front of a screen, and spend leisure activity time there too. Unfortunately, the effects of a high number of hours watching these lit pixels—albeit sometimes positive for moral or psychological aspects—are mostly considered negative for eyesight, causing eye fatigue, dry eyes, or difficulty focusing. On the other hand, there are some activities that will alleviate these ophthalmic conditions—most of them include simply doing something other than watching a screen. Usually, recommendations go along the path of regularly taking a stroll, reading a book, or engaging in physical activity.

It's never easy to pick up a new habit, and no one has ever gone from never jogging to running a marathon. The goal is always achieved incrementally. But the important point is to track how much of these habits we can develop in a week, and maybe adjust objectives for the next week.

In this chapter, we'll write a service in charge of registering such habits. The user will be able to create habits, give

them an expected frequency—a number of times per week they are expected to be experienced—and list them. We’ve already written an HTTP service, so this time we’ll focus on another popular network remote procedure call protocol developed by Google: gRPC. This project has the following requirements:

- Create and delete a habit.
- List the created habits.
- Tick a created habit.
- Get the status of a habit.

Additional technical requirements include the following:

- gRPC service, with Protobuf input and output
- Run locally
- In-memory database initially

10.1 API definition

In the same vein as we did in chapter 8, we’re going to create a web service to track personal habits, that is, a Go program that runs indefinitely, ready to listen to requests and respond to them. Requests are sent by clients, who need to know what to send and how to understand the response. Such a set of definitions is called an Application Programming Interface (API), which we’ve referred to here and there in this book. Here, the *clients* refer to users who want to track their habits. They will do it by calling the endpoints, such as `CreateHabit`, exposed on an API that we’re going to build.

In this case, we want the communication between the clients and our service to use the gRPC framework, where messages are encoded using the Protocol Buffers (Protobuf)

format and using the HTTP/2 network layer. Protocol Buffers are a programming language-independent description of how these messages are encoded.

PROTOCOL BUFFERS (PROTOBUF)

Protobuf fields are used for serializing structured data. While this can also be achieved in many other ways (JSON, XML, YAML, etc.), Protobuf emphasizes two important points: versioning the serialized model and reducing any non-data information. Invented by Google in 2001 and released to the public in 2008, Protobuf is perfect for low-latency, high-speed applications, such as microservices. Protobuf is a way to describe communication between programs in a cross-language way. You can define what data is being sent via message definitions. You can also define endpoints for what is being communicated. For messages and service APIs, the endpoints are written in Protobuf files (text files with the `.proto` file extension, usually), which can then be compiled to generate clients for the programming language of your choice, as we'll explain later. Clients can be generated for many common languages, including Go.

There are a few limitations, however: Protobuf messages aren't self-describing, so you need to know how to read them before you can access their contents. This also means we can't simply use regular tools such as `curl` to send messages to a gRPC endpoint. Testing will also be a bit trickier than with JSON APIs.

The first step in the development of a system, once we know the requirements, is generally to define the API, that is, how the system will be used. Any language can be used, as long as we know our users will use this language or that

some tools can be used to generate adequate files to connect to our servers. In this section, we'll use Protobuf to declare our API. Our Protobuf files will be compiled into Go files that we can then use to implement our service. The final API will resemble the following listing, and throughout this chapter, we'll go through each step necessary to implement these endpoints.

Listing 10.1 Habits service API

```
// Habits is a service for registering and tracking habits.
type HabitsService interface {
    // CreateHabit is the endpoint that registers a habit.
    CreateHabit(CreateHabitRequest) (CreateHabitResponse, error)

    // ListHabits is the endpoint that returns all habits.
    ListHabits(ListHabitsRequest) (ListHabitsResponse, error)

    // TickHabit is the endpoint to tick a habit.
    TickHabit(TickHabitRequest) (TickHabitResponse, error)

    // GetHabitStatus is the endpoint to retrieve
    // the status of ticks of a habit.
    GetHabitStatus(GetHabitStatusRequest) (GetHabitStatusResponse, error);
}
```

10.1.1 Protobuf declaration

While this isn't a book about Protobuf, we need a few basics to define our API. Initialize your go module the usual way by creating a directory and running the following:

```
> go mod init learn-go-pockets/habits
```

Even before creating a `main.go` or anything, create a folder at the root of the project, named `api/proto`, where we can store the Protobuf files that will have the `.proto` extension.

The service's job is to deal with habits, so create a file called `habit.proto` where we can define what a habit is. For the moment, we'll give it a name and a weekly frequency. For example, if you want to practice Go five times a week, you want to be able to send something along these lines:

```
{"practice Go", 5}
```

HABIT ENTITY

Let's start with a minimal API definition of what a habit is. It has a name and a weekly frequency. We can write a Protobuf file with the entity.

Each .proto file starts with the version of the protocol, then defines a Protobuf package, and in our case, because we want to generate Go code, a Go package. Generated code will end up in the folder named after the package and situated inside the go_package module path. As usual, a piece of code will make things clearer.

Listing 10.2 habit.proto: Headers of the proto file

```
syntax = "proto3"; #1  
package habits; #2  
option go_package = "learngo-pockets/habits/api"; #3
```

#1 Protobuf version

#2 Proto package name

#3 Go package path

Every structure in Protobuf is a *message*, and every field is given a number that will allow consumers to recognize it, as shown in listing 10.3. If we decide that the name is associated with 1, it will have to stay 1 forever, and future versions with different fields will still look for the name at index 1 to ensure backward compatibility.

Listing 10.3 habit.proto: Defining the Habit message

```
// Habit represents an objective one wants to complete
// a given number of times per week.
message Habit { #1
    // Name of the habit, cannot be empty
    string name = 1; #2
    // Frequency, expressed in times per week.
    int32 weekly_frequency = 2;
}
```

#1 Defines a structure, or message

#2 Field sequence starts at 1 .

In a Protobuf message, each field must have a unique identifier. There's no point in leaving gaps, so just follow the incremental order. The syntax is to list each field with its type followed by its name and identifier. You can find more examples and lists of supported types here:

<https://protobuf.dev/programming-guides/proto3/>.

Don't hesitate to be extremely verbose in your comments: this is what users will read to figure out how to use what you made, not the generated Go code. Comments in the .proto files will be carried over into the generated code.

SERVICE DEFINITION

Once we have this simple `Habit` object, we can declare a service to manipulate it. In another file, define a service that will use this message.

It's good practice, for version compatibility, to define a `Request` and a `Response`, even when they are empty or when they contain only one field: it makes changes smaller and avoids breaking the API. The gRPC `Habits` service is in charge of registering and tracking habits, as shown in the following listing. That is where we'll add, along the way, all the needed endpoints to track the habits.

Listing 10.4 service.proto: Defining the Habits service

```
syntax = "proto3";

package habits;
option go_package = "learngo-pockets/habits/api"; #1

// Habits is a service for registering and tracking habits.
service Habits { #2
}
```

#1 Same headers

#2 New service

FIRST ENDPOINT: CREATE

This service exposes nothing, as you can see. The first endpoint that we need is for creating a habit to track.

A generally accepted best practice when naming inputs and outputs of endpoints is to have a dedicated message (a structure with fields) for each of them, called `request` and `response`, or `input` and `output`, even when they are empty or contain only one field. Consider the difference between these two signatures:

```
func CreateHabit(Habit)
func CreateHabit(CreateHabitRequest) CreateHabitResponse
```

In the first case, we give a habit and expect nothing in return—simple. In the second case, we need to define two additional structures, and it's verbose and annoying. The first one will just contain a `Habit` field, and the other will be empty, so what's the point? The point is version intercompatibility. Let's say, in the next version, we want to add a user token to identify which user is creating the habit, and then return a habit identifier. In the more verbose case, we would just add a field in each structure, and if it isn't mandatory, any code written for the initial version will still work. In the first and straightforward case, however, we would break the whole API.

For this reason, the `CreateHabit` endpoint will use its `Request` and `Response`. You might wonder what happens in the case of errors: Why wouldn't they appear in the Protobuf API? The answer is that the gRPC compilation tool will be in charge of adding support for errors. This support differs from language to language—in Go, we can have several returned values, whereas in C++ or Java, the error needs to be returned differently, which means we don't write errors in the `.proto` file. Don't panic, though, the Golang interface compiled from this Protobuf will allow us to return an error. We can add the endpoint to the service with one line and then define two new messages.

Listing 10.5 `service.proto`: `CreateHabit` endpoint

```
service Habits {  
    // CreateHabit is the endpoint that registers a habit.  
    rpc CreateHabit(  
        CreateHabitRequest  
    ) returns (CreateHabitResponse); #1  
}
```

#1 Don't forget the semicolon.

To use the `Habit` message in the response, we need to import the neighboring file, as shown in listing 10.6. We could write the request structure and the service definition in the same file—but knowing how to import a `.proto` file is useful.

Listing 10.6 service.proto: Request and response

```
import "habit.proto"; #1

service Habits {
    ...
}

// CreateHabitRequest is the message sent to create a habit.
message CreateHabitRequest {
    // Name of the new habit. Cannot be empty.
    string name = 1;
    // Frequency of the new habit. Will default to once per week.
    optional int32 weekly_frequency = 2; #2
}

// CreateHabitResponse is the response of the create endpoint.
message CreateHabitResponse {
    Habit habit = 1;
}
```

#1 Imports file using the relative path

#2 Marks the field as optional

We're done. In a handful of lines, we have an API for the first step of the tracker, which is the creation of a habit. As you can see, there's no path and no verb: they are specific to HTTP. gRPC doesn't use them. Because we want to use it in Go, now is the time to generate the Go code.

10.1.2 Code generation

Generating code from Protobuf files is done using `protoc`, which is the Protobuf compiler. We'll also install two plugins: `protoc-gen-go` and `protoc-gen-go-grpc`. We'll cover the installation steps, but they are also provided at <https://mng.bz/MDz7>.

INSTALLATION STEPS

Depending on your system, installing `protoc` might be achievable simply through a package management tool such as Homebrew's `brew` (on Mac) or `apt` (on Linux). Unfortunately, there are a few more steps when installing it

on Windows. Here are the commands you can run from a terminal:

```
> apt install -y protobuf-compiler #Linux  
> brew install protobuf #Mac
```

Once `protoc` is installed, getting the Golang-specific dependencies is made easy by the fact that we can ask Go to do it:

```
> go install google.golang.org/protobuf/cmd/protoc-gen-go@latest  
> go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

These two utilities are used to compile `.proto` files declaring messages and services into Golang files. They work as plugins for `protoc`, and they will be called if `protoc` detects we want to compile Golang files.

COMPILATION

The compilation command is pretty long. We'll try to decompose it step by step. In your favorite terminal, navigate to the root of the `go` module, and try the very minimal version:

```
> protoc api/proto/habit.proto
```

The compiler complains because it needs output directives regarding what language to generate the compiled files in and where. The `go_out` parameter will tell the compiler both the requested output language and the location for compiled files by specifying the target folder. By specifying this option, we also tell `protoc` to use the `protoc-gen-go` plugin:

```
> protoc --go_out=api/ api/proto/habit.proto
```

This generates a `Habit` structure, but it puts the structure in an impractical location: the whole module tree is created all

over again:

```
> tree
.
├── api
│   ├── learnngo-pockets
│   │   └── habits
│   │       └── api
│   │           └── habit.pb.go
│   └── proto
│       ├── habit.proto
│       └── service.proto
├── go.mod
└── go.sum
```

That's not what we want; we want the Go file to appear directly in the `api` folder. Fortunately, there's an option for that: `--go_opt=paths=source_relative`, as shown here:

```
> protoc --go_out=api/ --go_opt=paths=source_relative
    api/proto/habit.proto
```

Now the tree looks like what we want. The last step is to compile all the `.proto` files, not just `habit.proto`:

```
> protoc --go_out=api/ --go_opt=paths=source_relative api/proto/*.proto
```

It doesn't work. The compiler is taking each file separately and generating a Go file for it. When it reaches `service.proto`, it can't import `Habit` because we never told it where to look.

The `-I` option has the following documentation if you run `protoc --help`: "Specify the directory in which to search for imports. May be specified multiple times; directories will be searched in order. If not given, the current working directory is used." This is perfect for our needs:

```
> protoc -I=api/proto/ --go_out=api/ --go_opt=paths=source_relative
    api/proto/*.proto
```

Once you've run this command, your tree should look like this:

```
> tree
.
├── api
│   ├── habit.pb.go
│   ├── proto
│   │   ├── habit.proto
│   │   └── service.proto
│   └── service.pb.go
├── go.mod
└── go.sum
```

All the messages exist as Go structures, but not the service yet. We also need to generate the gRPC part.

The options are quite similar to the pure Go ones: `go-grpc_out` and `go-grpc_opt`. Passing these options on the command line will silently tell `protoc` to use the `protoc-gen-go-grpc` plugin:

```
> protoc -I=api/proto/ --go_out=api/ --go_opt=paths=source_relative
--go-grpc_out=api/ --go-grpc_opt=paths=source_relative
api/proto/*.proto
```

There's one final parameter that we must talk about when it comes to the Go gRPC compiler, and this has to do with forward compatibility. Suppose that we're happy with the current `.proto` API, that we use it to compile the Golang files, and that we implement the server interface with a structure of our own. Then, let's assume we want to add a new endpoint—we'll have to update the `.proto` file and regenerate the Golang files. As mentioned on the `go-grpc` repository, "It is a requirement that adding methods to a service can't break existing implementations of the service." So, how did they ensure this requirement is always met?

There are two options. The first one is to require that any implementation of the server embeds a type defined in the generated file. The other is to allow for the developers to not implement the required server interface. While this second option isn't recommended, it's still available by passing another parameter to the command line:

```
--go-grpc_opt=paths=source_relative,require_unimplemented_servers=false
```

In the rest of this chapter, we'll use files that were generated without this final option, and we'll remind you to embed the type when creating the server type.

AUTOMATED GENERATION

Remember to put this massive command in a place where you and future maintainers will find it, typically in a `Makefile` or as part of a script. You might wonder why we wouldn't place this in a `generate.go` file containing only a `//go:generate` directive (writing this is a nice trick to keep up our sleeve). The reason is that we were lazy in our command line and used a `*` to send all the `.proto` files to `protoc`. Unfortunately, while shells understand the string `*.proto` as every file with a `proto` expansion, `go generate` doesn't, which prevents us from using the same command line directly in a `//go:generate` directive. However, if you have access to `bash` or `sh`, or any other shell you fancy, you can tell `go generate` to run a command in a shell with the following syntax (don't forget the double quotation marks around the command that you really want to run):

```
//go:generate bash -c "protoc -I=api/proto/ {...} api/proto/*.proto"
```

We provided an example of a `generate.go` (a common name for files that only contain `//go:generate` directives) file that contains a similar command. We slightly adapted it because we placed it directly into the `api` directory. It's up to you to decide whether you want a target in your `Makefile` or if you'd rather call `go generate` to produce these files.

Make sure to document it, like everything that isn't considered general knowledge in the industry. Your tree should now look something like this:

```
> tree
.
├── api
│   ├── proto
│   │   ├── habit.proto
│   │   └── service.proto
│   ├── generate.go
│   ├── habit.pb.go
│   ├── service_grpc.pb.go
│   └── service.pb.go
├── go.mod
└── go.sum
```

Ready to start coding in Go? Let's get to it.

10.2 Empty service

Now that we have an API exposing the create-habit endpoint for our user, we can write the code and make it run. We'll first create an empty service, make it run, and then add the endpoints. After that will come the data layer, and, finally, integration tests, and we'll be ready to start again with more functionalities.

10.2.1 Creating a small logger

A logger is often the first package that is written in a module, as it will likely be used by every other package. But loggers can sometimes be problematic—they'll write to whatever output we tell them to write to. Sometimes, this causes problems, for instance: Should the log messages always be printed when testing? And to what output? In this section, we'll implement a small logger that will make it easier for us to both run and test our code with logs.

Notice that the `testing.T` structure already implements a `(t *T) Logf(format string, args ...any)` method that will only print what we called it with when the current test fails. To be able to use a logger in our code and in our tests, let's write a small logger that will only expose one method—the same as

exposed by `testing.T`, as shown in the following listing. This way, we'll be able to create loggers in our code and inject the `t` test variable in tests as the test logger. This will prevent output jamming.

Listing 10.7 `log/log.go`: Defining a small logger

```
package log

import (
    "io"
    "log"
    "sync"
)

// A Logger that can log messages
type Logger struct {
    mutex sync.Mutex #1
    logger *log.Logger
}

// New returns a logger.
func New(output io.Writer) *Logger {
    return &Logger{
        logger: log.New(output, "", log.Ldate|log.Ltime), #2
    }
}

// Logf sends a message to the log if the severity is high enough.
func (l *Logger) Logf(format string, args ...any) {
    l.mutex.Lock()
    defer l.mutex.Unlock()
    l.logger.Printf(format, args...)
}
```

#1 We don't know if writing to the output is concurrency safe, so we enforce it.

#2 Adds some niceties to the default log package

Now that we have our basic logger, we can start implementing the server package. We'll need this logger in each of our packages.

10.2.2 Server structure

First, we create a structure that will be our server. It wouldn't make sense if our server structure were to stay empty for long; it will soon contain a repository for data retention.

In a new folder named `internal/server`, create a `server.go` file, and add the struct with a `New` function. As we'll want to use a logger, let's declare a one-method interface that we'll use as our logger.

Listing 10.8 `server.go`: Defining the web service

```
// Server is the implementation of the gRPC server.
type Server struct {
    lgr Logger #1
}

// New returns a Server that can ListenAndServe.
func New(lgr Logger) *Server { #2
    return &Server{
        lgr: lgr,
    }
}

type Logger interface {
    Logf(format string, args ...any)
}
```

#1 Uses the local interface

#2 Injects the logger dependency

Next, we need to add a `ListenAndServe` method on the `Server`, so that it can start listening to and serving new requests sent on a given port. Ports are virtual “doors” through which messages transfer, either internally or with the rest of the world, so that only one application can listen to a given port on the same machine. Ports are identified with their port number—80 is used by HTTP, 443 by HTTPS, and so on. When listening to a specific port, either use one that is assigned by a standard, such as 80 for HTTP, or use a port number between 1024 and 49151 for internal usage.

A gRPC server, just like the HTTP server we saw in chapter 8, is first and foremost a good listener. We give it a port to listen to, and start it with a call to `Serve`. This call will only return when the server shuts down.

But a gRPC server is a bit more than a HTTP server—it must implement the desired gRPC API. For this, we start by

creating a barren server using the `grpc` package, and we then attach our implementation to that server by registering it with the generated `RegisterHabitsServer` function of the `api` package, as shown in the next listing. Registering a server associates a specific service implementation to a generic gRPC server.

Listing 10.9 `server.go`: Listening to a given port

```
import (
    ...
    "google.golang.org/grpc"
    "learn-go-pockets/habits/api"
)

// ListenAndServe starts listening to the port and serving requests.
func (s *Server) ListenAndServe(port int) error {
    const addr = "127.0.0.1"

    listener, err := net.Listen("tcp",
        net.JoinHostPort(addr, strconv.Itoa(port))) #1
    if err != nil {
        return fmt.Errorf("unable to listen to tcp port %d: %w", port, err)
    }

    grpcServer := grpc.NewServer() #2
    api.RegisterHabitsServer(grpcServer, s) #3

    s.lgr.Logf("starting server on port %d\n", port)

    err = grpcServer.Serve(listener) #4
    if err != nil {
        return fmt.Errorf("error while listening: %w", err)
    }

    // Stop or GracefulStop was called, no reason to be alarmed.
    return nil
}
```

#1 Uses the given port with TCP

#2 Creates a standard gRPC server

#3 Registers our methods in it

#4 Starts listening

There are better ways of starting the server to support graceful shutdown. We'll improve this later in the chapter. Additionally, if you want to allocate a free port randomly, you can use port 0. The documentation of `net.Listen` explains which networks are supported.

Wait—this doesn't compile. We can't register a `HabitService` that doesn't know how to create a habit. As you can see, `api.RegisterHabitsServer` takes as a second parameter anything that implements the `HabitServer` interface, which was generated from our Protobuf service. We just need to implement that one method.

When trying to compile or run, we also faced an error mentioning that our `Server` type can't be registered as a `HabitsServer` because it doesn't implement a method named `mustEmbedUnimplementedHabitsServer`. This is a reminder that when we generated the Go files from the `.proto` files, we used the recommended way, which requires embedding a structure, as the non-implemented method's name suggests. So, let's embed the required type:

```
// Server is the implementation of the gRPC server.
type Server struct {
    api.UnimplementedHabitsServer
    lgr Logger
}
```

COMPOSITION AND EMBEDDING

Both the composition and embedding concepts extend the notion of a structure, but in a different way. While composition, which in Go is achieved by listing named fields of a structure, represents a “has-a” relationship between two types, embedding corresponds to an “is-a” relationship.

In our case, our server is an `UnimplementedHabitsServer`, so it has an implementation for that required method. You can read more on the Go dev blog (https://go.dev/doc/effective_go#embedding).

As we know that this method will require tests and probably side functions, we can already put it in a `create.go` file in the `server` package. The signature of this function was generated by the `protoc` toolchain; we can't alter it. In the following listing, there's a mysterious first parameter, which we'll dive into later in section 10.3.

Listing 10.10 `create.go`: Implement the `HabitServer` interface

```
// CreateHabit is the endpoint that registers a habit.
func (s *Server) CreateHabit(
    _ context.Context, #1
    request *api.CreateHabitRequest
) (*api.CreateHabitResponse, error) {
    s.lgr.Logf("CreateHabit request received: %s", request)

    return &api.CreateHabitResponse{
        Habit: &api.Habit{}, #2
    }, nil
}
```

#1 Ignores the context for now

#2 For now, returns an empty response

This should be enough for now. We'll come back to it very quickly. Our endpoint's scaffolding is implemented—our whole gRPC layer is implemented. It's now time to spin up the service.

10.2.3 Creating and running the server

We can call these `New` and `ListenAndServe` functions in `main`. As this is a web service, we prefer to put the `main.go` file in a `cmd/habits-server` folder, decluttering the root of the module. On some operating systems, such as Windows, `go run dir/main.go` will cause an executable file called `dir.exe` to be generated and executed, so placing the `main.go` file in an aptly named directory is important in that regard.

The `main` function creates a new instance of our server and calls `ListenAndServe`, which only returns if there's an error. Because we need to inject a logger into our server instance,

we can create it in the `main` function and pass it via `server.New`. We can use that logger in the `main` function too. The following listing shows how we create a new server in our `main` package and run it.

Listing 10.11 `main.go`: Running it

```
package main

import (
    "fmt"
    "os"

    "learngo-pockets/habits/internal/server"
    "learngo-pockets/habits/log"
)

const port = 28710 #1

func main() {
    lgr := log.New(os.Stdout) #2

    srv := server.New(lgr) #3

    err := srv.ListenAndServe(port) #4
    if err != nil {
        lgr.Logf("Error while running the server: %s", err.Error())
        os.Exit(1) #5
    }
}
```

#1 Declares a port in range 1 024-49 1 5 1

#2 Sets the new logger output

#3 Creates a new server

#4 Runs the server

#5 Exits with an error code

There's basically no logic inside the `main` function. This means that our service will be easier to test because all the logic is in isolated packages. Run it as follows:

```
> go run cmd/habits-server/main.go
```

It does absolutely nothing, but it runs, which is wonderful! Add a few logs in `ListenAndServe` to make sure.

10.3 First endpoint: Create

We have a running gRPC server that implements the desired API. We might want to go a step beyond and have our endpoint do something other than print a pretty message. This is, after all, chapter 10.

Before we start coding, let's do a bit of thinking. The compiled code, generated from the Protobuf files, has defined a `Habit` structure. Should we reuse that structure or define a new one? The answer is quite straightforward here: it's always best to not leak protocol definitions into the core business code because it will create problems when we start to add support for other protocols, such as XML or JSON. These data definitions that are used only for describing protocols are called data transfer objects (DTOs). Instead, our core business code, often called “domain” or “model,” should have types for every entity that needs to be handled internally. Let's look at a clean target architecture in figure 10.1.

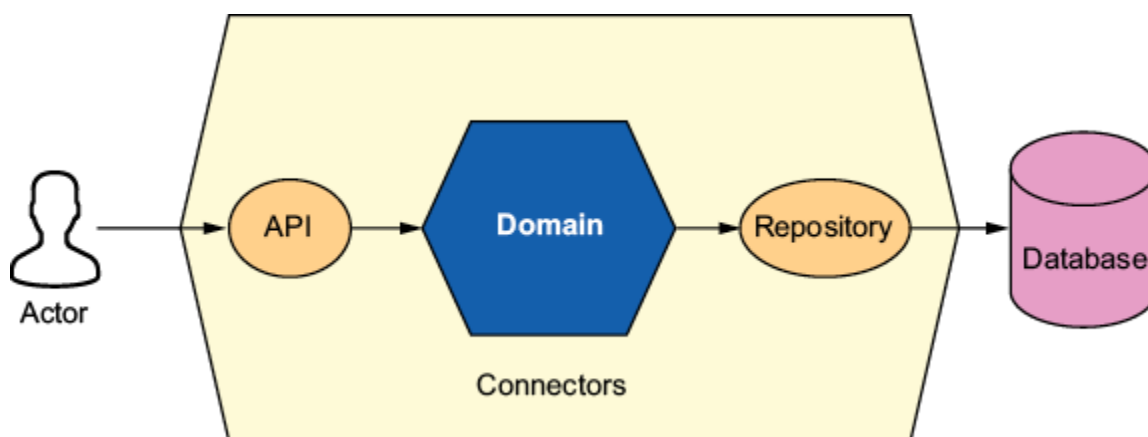


Figure 10.1 Architectural diagram with domain and connectors

For the same reasons that we saw in chapter 8, the Go structures representing the transferable data, here the

generated code, must be capable of evolving independently from the rest of the code.

10.3.1 Business layer

In chapter 8, we created a `session` package with our logic. Create an `internal/habit` folder where we can define our domain `Habit` and the types that it needs, as shown in listing 10.12. We want to keep the data that was received as input but also remember when the habit was created and give it an ID to find it again. These fields aren't part of the input message—we're able to add them here because we're not reusing the API structure.

Listing 10.12 `internal/habit/habit.go`: Defining the business types

```
// ID is the identifier of the Habit.
type ID string

// Name is a short string that represents the name of a Habit.
type Name string

// WeeklyFrequency is the number of times a Habit should happen every week.
type WeeklyFrequency uint

// Habit to track.
type Habit struct {
    ID          ID #1
    Name        Name
    WeeklyFrequency WeeklyFrequency
    CreationTime time.Time #1
}
```

#1 Fields internal to the domain

It's always good to create a specific type for each of the fields in our main entity, even though the usage might seem more verbose: functions and methods will take typed arguments that will serve as documentation and make the API clearer. For example, if a function takes the name and the ID, and both are strings, it's quite easy to mix them up. On the other hand, if one is explicitly an ID and the other explicitly a name, casting the name into an ID type should raise a red flag to the developer writing the call.

REQUIREMENT: CREATE A HABIT

If we look at the requirements, the first thing we need to do is create a `Habit`. We defined some optional values in the Protobuf documentation, meaning that we must complete the fields if needed. First, we must determine if input validation is the job of the API layer or the domain layer. Both solutions make sense for different reasons. We decided that if some new feature needs to create a habit inside our service, it will call the domain directly, and we want this `NewHabit` to always return a valid entity. We can't rely on the API layer to always send what the domain needs.

Tip If you're in a situation where validation on the API layer makes more sense, there are a few libraries out there, such as the package validator at <https://github.com/go-playground/validator>, that can do it for you with a few tags.

What if the input is invalid? Just like HTTP, gRPC uses different status codes, as shown in table 10.1, to make sense of the response defined by the RPC API. These codes are included in the error that is returned alongside the response, by the endpoint.

Table 10.1 A few gRPC status codes

Code	Number	Description
OK	0	This isn't an error; it's returned on success.
INVALID_ARGUMENT	3	The client specified an invalid argument.
NOT_FOUND	5	Some requested entity (e.g., file or directory) wasn't found.
PERMISSION_DENIED	7	The caller doesn't have permission to execute the specified operation.
UNIMPLEMENTED	12	The operation isn't implemented or isn't supported/enabled in this service.
INTERNAL	13	An unspecified error occurred while processing the request.

These are only a few; the rest, with deeper explanations, can be found in the official documentation. You can run `go doc google.golang.org/grpc/codes` to get the list. Well, that's not exactly true because `go doc` limits its output, which causes only the first few codes to be printed. To get the whole list, run the following:

```
> go doc --all google.golang.org/grpc/codes
```

With these codes in mind, we know that any invalid input will return a code 3 (INVALID_ARGUMENT). The business layer certainly shouldn't be in charge of returning such a code 3. Returning this code is the role of the API layer—the domain layer isn't even aware that we're implementing a gRPC

server—and it needs to know what happened inside the domain layer. This is the perfect occasion to use a typed error.

VALIDATE WITH TYPED ERROR

Let's start with the `validateAndComplete` function in a new `create.go` file dedicated to this business logic, as shown in listing 10.13. It must check that the name isn't empty, set the frequency to 1 if empty, and also fill up the two internal fields. Arguably, it could be two functions: `validate` and `complete`.

Listing 10.13 `internal/habit/create.go`: **Validate/complete entity**

```
// validateAndCompleteHabit fills the habit with values that we want in
// our database.
// Returns InvalidInputError. #1
func validateAndCompleteHabit(h Habit) (Habit, error) {
    // name cannot be empty
    h.Name = Name(strings.TrimSpace(string(h.Name))) #2
    if h.Name == "" {
        return Habit{}, InvalidInputError{ #3
            field: "name", reason: "cannot be empty"}
    }

    if h.WeeklyFrequency == 0 {
        h.WeeklyFrequency = 1 #4
    }

    if h.ID == "" {
        h.ID = ID(uuid.NewString()) #5
    }

    if h.CreationTime.Equal(time.Time{}) {
        h.CreationTime = time.Now()
    }

    return h, nil
}
```

#1 Tells readers what to expect from your function

#2 A row of spaces is invalid too.

#3 Returns a structured error

#4 Defaults to 1

#5 A new unique ID

We now need to define this typed error, in a new `errors.go` file in the `habit` package.

Listing 10.14 `errors.go`: Typed error for invalid input

```
// InvalidInputError is returned when user-input data is invalid.
type InvalidInputError struct {
    field string
    reason string #1
}

// Error implements error.
func (e InvalidInputError) Error() string {
    return fmt.Sprintf("invalid input in field %s: %s", e.field, e.reason)
}
```

#1 A short description of why the provided value is invalid

We could expose the given value of the field too: it's very useful when we get an error to know what the server actually got, as it differs from what we think we sent more often than we care to admit. But this error will be logged and copied around, and malevolent users could send in gigabytes of data and crash our system. There are some ways to avoid this (limiting the size of requests, truncating logs, etc.), but for now, let's just avoid logging the field.

TESTING THE VALIDATION

We can already write an easy unit test for this `validateAndComplete` function. There's no need to mock any dependency—what a pleasure!

When writing this test, we found out that each test case had very different assertions, so we chose to write a named function for each. You can write several independent `TestXxx` functions or group them inside a single one with an explicit name.

Listing 10.15 create_internal_test.go: Testing completeHabit

```
func Test_validateAndFillDetails(t *testing.T) {
    t.Parallel()

    t.Run("Full", testValidateAndFillDetailsFull)
    t.Run("Partial", testValidateAndFillDetailsPartial)
    t.Run("SpaceName", testValidateAndFillDetailsSpaceName)
}
```

The first function, `testValidateAndFillDetailsFull`, checks that if the habit is complete, nothing is changed:

```
func testValidateAndFillDetailsFull(t *testing.T) {
    t.Parallel()

    h := Habit{/* all fields are filled */}

    got, err := validateAndCompleteHabit(h)
    require.NoError(t, err)
    assert.Equal(t, h, got)
}
```

The second function checks that if the habit is incomplete, ID and creation time are filled up, and the rest didn't change. Each run of the test will give us different values, so we're only certain that these fields are at least using the `NotEmpty` function. If you want to be more thorough, you can check that the ID follows a given format using regular expressions and that the time is within the past second or so:

```
func testValidateAndFillDetailsPartial(t *testing.T) {
    t.Parallel()

    h := Habit{/* only Name and Freq */}

    got, err := validateAndCompleteHabit(h)
    require.NoError(t, err)
    assert.Equal(t, h.Name, got.Name)
    assert.Equal(t, h.WeeklyFrequency, got.WeeklyFrequency)
    assert.NotEmpty(t, got.ID)
    assert.NotEmpty(t, got.CreationTime)
}
```

Then, we check the name. If we send only spaces, we should get an `InvalidInputError`. At this point in the

development, the exact content of the error might still change, so we just focus on the type:

```
func testValidateAndFillDetailsSpaceName(t *testing.T) {
    t.Parallel()

    h := Habit{Name:"    "}

    _, err := validateAndCompleteHabit(h)
    assert.ErrorAs(t, err, &InvalidInputError{})
}
```

That's good. Run the test to make sure you're happy about your coverage.

Now, let's call the endpoint. The `create` function in the business (or domain) layer will fill the habit and be ready to save it to a data storage.

Listing 10.16 `create.go`: Business function to create a habit

```
// Create validates the Habit, saves it and returns it.
func Create(_ context.Context, h Habit) (Habit, error) {
    h, err := validateAndFillDetails(h)
    if err != nil {
        return err #1
    }

    // Need to add the habit to data storage...

    return h, nil
}
```

#1 Wraps it if you want some context

You can already write a closed-box test for this one, or at least the structure for the test. We added a `context` as the first parameter, but we've ignored it until now. Why? The answer to this is quite simple, and we'll explain it fully in section 10.6. For now, let's provide a `context.Context` variable, which, in Go, is almost always called `ctx`.

10.3.2 API layer

Now that we've implemented the validation in the domain layer, let's move back to the server package and update the `CreateHabit` method on the server structure, as shown in listing 10.17. This is the gRPC layer, where we transform an API-specific signature into domain objects, call the domain function, and transform the response back into API-specific types.

Listing 10.17 create.go: API layer

```
// CreateHabit is the endpoint that registers a habit.
func (s *Server) CreateHabit(ctx context.Context,
    request *api.CreateHabitRequest) (*api.CreateHabitResponse, error) {
    var freq uint #1
    if request.WeeklyFrequency != nil {
        freq = uint(*request.WeeklyFrequency)
    }

    h := habit.Habit{
        Name:          habit.Name(request.Name),
        WeeklyFrequency: habit.WeeklyFrequency(freq),
    }

    createdHabit, err := habit.Create(ctx, h)
    if err != nil {
        ...
    }

    s.lgr.Logf("Habit %s successfully registered", createdHabit.ID)

    return &api.CreateHabitResponse{
        Habit: &api.Habit{
            Id:          string(createdHabit.ID),
            Name:        string(createdHabit.Name),
            WeeklyFrequency: int32(createdHabit.WeeklyFrequency),
        },
    }, nil #2
}
```

#1 Initializes with default value 0

#2 Doesn't return { ... }, `status.Error(codes.OK, "everything went fine")`

If we want the default value of a habit's frequency to be 1, why are we setting the `freq` to 0 (by using Go's default value) when it's absent? We decided that this default value was a business requirement and not an API definition. It's

arguable and can only be decided on a case-by-case basis. Imagine what you expect if you call the domain method with an empty frequency from somewhere other than the API layer, and act accordingly.

How do we manage the error returned by the domain layer? We made sure that if the error is caused by a bad input, it will have a specific type. We can use `errors.As` to cast it into the `InvalidInputError` type and check whether we should return a code 3. To be perfectly honest, we could use `errors.Is` instead because we're not using any field or method specific to the type `InvalidInputError`, but we chose to show you how `As` can be used.

But what if we receive something that isn't an `InvalidInputError`? After all, our future implementation of the endpoint logic might have to face database calls, which could cause errors that wouldn't be due to an input message validation.

A rule of thumb to remember when implementing a gRPC endpoint is that every return statement should either return a `nil` error or an error built from the `status.Error` function (or `Errorf`). The `status` package is a neighbor of the `codes` package: google.golang.org/grpc/status. When in doubt about which error code we should return, the default choice is `codes.Internal`, as shown in the following listing.

Listing 10.18 `create.go`: Error management

```
got, err := habit.Create(ctx, h)
if err != nil {
    var invalidErr habit.InvalidInputError
    if errors.As(err, &invalidErr) {
        return nil, status.Error(codes.InvalidArgument, invalidErr.Error())
    }
    // other error
    return nil, status.Errorf(codes.Internal, "cannot save habit %v: %s",
        h, err.Error())
}
```

When a service ends up having several endpoints, checking the error and outputting the appropriate status code can be factored in a single function, `toAPIErrorf(err error, format string, args ...any)`. Feel free to implement it when the need arises.

Now it's time to test our server manually. The tool we used in chapter 8 to call our service, `curl`, only does HTTP calls, but it has a cousin, `grpcurl`, which does the same job. There are alternative options to `grpcurl`—many providing a GUI, but this one is the one we find most convenient. If you fancy a nice GUI, Postman supports gRPC and has been able to send Protobuf messages to servers since version 10.

First, start your server with `go run cmd/main.go`. Next, you can install `grpcurl` with the following command:

```
> go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest
```

Now, we can start using the tool to send requests to our server. There's a major difference between `curl` and `grpcurl`: the format of the message was, for `curl`, a regular JSON document, whereas for `grpcurl`, we need to provide a valid Protobuf entity. If you remember the beginning of this chapter, Protocol Buffers have indexed fields, which means the message we'll send via `grpcurl` will need to be properly written, with its fields in the correct positions. There are two options for us here—we can either provide the Protobuf definition to `grpcurl` or have it ask for that definition from the server. The second option is called *reflection*, and we won't be using it here. Reflection adds a small overhead to our server—something that usually we don't want to ship to production.

To tell `grpcurl` how to structure our query (and understand the response), we simply pass it the `.proto` files with the `-proto` parameter—we'll give it the `service.proto` file, as this is

where the definition of the endpoints lie. Because some of the files include other files, we need to specify the root from which they refer via the `-import-path` parameter. Finally, we need to tell which endpoint we want to aim at. This is passed as the final parameter of the request in the form of `{package}.{service}/{endpoint}`. Here's the command line that we're able to run:

```
> grpcurl \
  -import-path ../../learngo-pockets/habits/api/proto/ \ #1
  -proto service.proto \ #2
  -plaintext -d '{"name":"clean the kitchen"}' \ #3
  localhost:28710 \ #4
  habits.Habits/CreateHabit #5
```

#1 Absolute path to the proto directory

#2 File declaring the service

#3 Provides the request in JSON format

#4 Host and port where the server runs

#5 Target endpoint

If everything went fine, you should receive a response from the server (formatted in JSON). Does it contain an ID field? Is the weekly frequency set? Did you also try with an “invalid” name for the habit?

Check the server's output—it should be logging a message every time a request is received. If your tests are conclusive, it's time to do a bit more than logging in our endpoint!

10.3.3 Data retention

The service tells its clients that it can create habits, but it doesn't store them. We need to fix that. There are several options for data retention—the most common one being using a database. Appendix G shows how to connect to a database.

REPOSITORY PACKAGE

For the first version, we can use the same kind of in-memory repository that we used for games in chapter 8 in a package called `internal/repository`. It has the same drawbacks of being unscalable and likely soon unstable, but it gives us something quickly, so it's OK for a proof of concept.

Write a `Repository` structure with a `New` function that builds it and initializes its `map` of data. Similar to the `New` function of the `server` package, we want to inject a `Logger` in here too. For now, we'll need one `Create` method on the `Repository` type, but soon we'll want to add `List`, which will return all the contents of our database.

If you've followed us through nine chapters, you should be able to create the package; expose the right functions, structures, and methods; and of course cover them with some tests. Don't forget to add a mutex to lock the data when reading and writing on the repository storage:

```
> go doc
package repository // import "learngo-pockets/habits/internal/repository"

Package repository accesses the habits data.

type Error string
type Logger {...}
type HabitRepository struct{ ... }
    func New(lgr Logger) *HabitRepository

> go doc HabitRepository
package repository // import "."

type HabitRepository struct {
    // Has unexported fields.
}
    HabitRepository holds all the current habits.

func New(lgr Logger) *HabitRepository
func (hr *HabitRepository) Create(_ context.Context, habit habit.Habit) error
func (hr *HabitRepository) ListAll(_ context.Context) ([]habit.Habit, error)
```

By having a `context.Context` parameter in each of our methods, we anticipated that when we replace this with a real database, we'll need a `context` to stop looking for data when a client interrupts the call. If you need it, remember that an example of the code can be found in the book's repository (<https://mng.bz/nRj2>).

DEPENDENCY INJECTION

We didn't really test this `repository` package. The main reason here is that all we do in this package is write to a `map` and list all values of a `map`. We can add the call to `Add` inside the domain function.

We've implemented the API-to-domain connection in the `server` package, but we lack the right-hand calls. For that, we need the server to have an instance of the `Repository` connector, and we need the domain's `Create` function to expect a small interface with `Add` in it.

Let's first inject a repository dependency to the server, as shown in listing 10.19. But why don't we simply call `repository.New()` in the server, rather than doing it in the `main` function? As we'll see, this makes tests a lot simpler than having to rely on a hardcoded implementation of that dependency. This is one of Go's best usages of its lightweight interfaces. We're using an interface here so that tests for the server can use mocks.

Listing 10.19 server.go: Adding a repository connector to the server

```
// Server is the implementation of the grpc server.
type Server struct {
    db Repository
    lgr Logger
}

type Repository interface { #1
    Add(ctx context.Context, habit habit.Habit) error
    FindAll(ctx context.Context) ([]habit.Habit, error)
}

// New returns a Server that can Listen.
func New(repo Repository, lgr Logger) *Server {
    return &Server{
        db: repo,
        lgr: lgr,
    }
}
```

#1 Use the smallest possible interface we need.

First, Update the `main` function to comply with this new signature of `New`. We need to pass an entity that implements the interface, such as the output of `repository.New(...)`.

Second, the `Create` endpoint on the domain needs to take an interface as parameter (see listing 10.20) for stubbing and mocking purposes as well as to reduce the scope of problems. By using an interface with only the `Add` method, we ensure that `Create` can't use any other future method and mess with the logic. Imagine you observe in your logs that calls to `FindEverything` are messing with the performance of the service. You know by seeing this interface that the culprit isn't `Create`, and you can move on.

Listing 10.20 create.go: Calling the repository in the logic function

```
type habitCreator interface {
    Add(ctx context.Context, habit Habit) error
}

// Create adds a habit into the DB.
func Create(ctx context.Context, db habitCreator, h Habit) (Habit, error) {
    h, err := validateAndCompleteHabit(h)
    if err != nil {
        return Habit{}, err
    }

    err = db.Add(ctx, h)
    if err != nil {
        return Habit{}, fmt.Errorf("cannot save habit: %w", err)
    }

    return h, nil
}
```

Can you see in the logs that your call goes all the way to the database? Let's write a couple of tests to ensure we properly catch the errors. For this, we'll start with a simple stub, as we did in chapter 6, to implement the `habitCreator` interface.

But how can we update the tests of `Create` to make sure that `Add` is properly called? That's what we're going to see in the next section.

10.4 Unit testing with generated mocks

In the preceding chapters, you've seen how to write your own stubs when the interface is small enough and the logic is simple. There are a few libraries out there capable of taking an interface and generating mocks for you.

Definition Stubbing and mocking are two very common ways of making use of an interface for tests. While *stubbing* consists of writing a structure that implements the interface and returns hardcoded values, to test the

behavior of your code when the stubbed dependencies returns this or that, *mocking* adds a check of how many times each dependency was called and if it was done with the correct parameters.

10.4.1 Generate mocks

The best known libraries are `mockgen`, `mockify`, and `minimock`. They are based on different design decisions, so feel free to pick your favorite. In our example, we chose `minimock` because it provides mocked functions with typed parameters. Install `minimock` with the following:

```
> go install github.com/gojuno/minimock/v3/cmd/minimock@latest
```

Because the mocks are generated, this is a perfect occasion to use the `go:generate` syntax. Pick an interface, for example, `habitCreator`, and add the following line right before the interface's declaration:

```
//go:generate minimock -i habitCreator -s "_mock.go" -o "mocks"  
type habitCreator interface {  
    ...
```

We're asking `minimock` to generate a mock for the interface (`-i`) `habitCreator` in a file with a specific suffix (`-s`) and in a specific output folder (`-o`). You need to create that folder before you can continue: from the root of the module, it will be `internal/habit/mocks`. In your favorite terminal, navigate to the `habit` package and run the following:

```
> go generate .
```

Alternatively, you can navigate to the root of the module and run all the generate commands in the project with

```
> go generate ./...
```

You can see that a new file has appeared in the `mocks` folder. Check the contents with `go doc`:

```
> go doc internal/habit/mocks
package mocks // import "learn-go-pockets/habits/internal/habit/mocks"

type HabitCreatorMock struct{ ... }
    func NewHabitCreatorMock(t minimock.Tester) *HabitCreatorMock
type HabitCreatorMockAddExpectation struct{ ... }
type HabitCreatorMockAddParams struct{ ... }
type HabitCreatorMockAddResults struct{ ... }
```

10.4.2 Use the mocks

The closed-box test for `Create` doesn't compile anymore, so let's fix it. First, there are two imports that we need to add. One is pretty obvious, but the second calls for a little explanation:

```
import (
    // ...
    "learn-go-pockets/habits/internal/habit/mocks"
    "github.com/gojuno/minimock/v3"
)
```

The first import is here to access the mocks we just generated. The second, on the other hand, is about the `minimock` library.

If you pay extremely close attention, you'll realize that the second import's path ends with `/v3`. Are we really importing a package named `v3`? This would be a very strange name for a package.

VERSIONING MODULES IN GO

Sometimes, a module needs to go through heavy changes that make the new version incompatible with the previous one. Interrupting the backward compatibility of a module requires a version change. When this happens, the `go.mod` file should be updated to reflect the version: the first line of `minimock`'s `go.mod` is `module github.com/gojuno/minimock/v3`.

Users who want to use `minimock` (or any other versioned module) have to remember to specify the version they want to use in the import path, right after the name of the module, for instance: `import "github.com/jackc/pgx/v5"` and `import "github.com/jackc/pgx/v5/pgxpool"`. When using functions or types defined in these packages, ignore the `"/v5"` part: `pgx.Connect(...)` or `pgxpool.New(...)`.

We need to use the `minimock` library because the generated code doesn't contain enough tools. In fact, the generated `mocks` package exposes a `NewHabitCreator(...)` function, which returns the type we want—`HabitCreatorMock`, an implementation of the `habitCreator` interface. But this function's parameter is of type `*minimock.Controller`. Don't worry, there's no other package to import.

Next, we can now define a function that builds a mock for each of the test cases. It takes a controller and returns a mocked instance of the required interface, `habitCreator`, as shown in listing 10.21. The test case structure will hold a new field whose type is a function—to be honest, if you look at the test with error cases that we have in the book's repository (<https://mng.bz/8OYz>), you'll see that it would be far easier to read if we had written it as two separate

functions, but we wanted to show you how functions can make your life better as fields of a test case struct.

The controller is created at the start of the test, and it can be shared by all the test cases. In addition, if your version of minimock is recent enough (v3.3.0), it automatically registers a check at the end of the test to ensure each expected call was met with an actual call. In the nominal test case, or happy flow, the mock should take the input habit, previously declared as a variable called `h`, and return no error.

Listing 10.21 `create_test.go`: Adding a mock function to each test case

```
tests := map[string]struct {
    db      func(
        *minimock.Controller) *mocks.HabitCreatorMock
    expectedErr error
}{
    "nominal": {
        db: func(ctl *minimock.Controller) *mocks.HabitCreatorMock {
            db := mocks.NewHabitCreatorMock(ctl)
            db.AddMock.Expect(ctx, h).Return(nil) #1
            return db
        },
        expectedErr: nil,
    },
}
```

#1 Registers the expected call

Finally, we can plug this into the `TestCreate` function.

Listing 10.22 `create_test.go`: Using the mock when testing

```
t.Run(name, func(t *testing.T) {
    t.Parallel()

    ctrl := minimock.NewController(t)
    defer ctrl.Finish() #1

    db := tt.db(ctrl)

    got, err := habit.Create(ctx, db, h)
    assert.ErrorIs(t, err, tt.expectedErr)
    if tt.expectedErr == nil {
        assert.Equal(t, h.Name, got.Name)
    }
})
```

#1 Versions >= 3.3.0 of minimock no longer require this line.

The code runs and succeeds. You can commit to make sure you don't forget this state, before playing around with the mocks. For example, what happens if you comment out the call to `Add`? Your test should tell you.

You can also read the documentation of the `mocks` package and how the `minimock` tool can best be used to define your favorite style. There's one more way of testing that is perfect for this kind of CRUD (create, read, update, and delete) service: integration testing.

10.5 Integration testing

Whereas unit tests focus on one function, integration tests will check the behavior of the entire service with scenarios. Here are example scenarios for our service:

- *Scenario 1: Add and list*
 - a. Add a habit: walk in the forest three times a week
 - b. Add a habit: water the plants twice a week
 - c. List the habits: check the two habits that we get
 - d. Add a habit: read a book five times a week
 - e. List the habits: check the three habits that we get
- *Scenario 2: Add and delete*
 - a. Add a habit: walk in the forest three times a week
 - b. List the habits: check the one habit that we got
 - c. Add a habit: no name, expected error with code 3
 - d. Add a habit: water the plants twice a week
 - e. List the habits: check the two habits that we got
 - f. Remove the first habit
 - g. List the habits: check that we still have the second

And there can be many more scenarios, of course. Some people will intertwine this with API testing; others will separate testing the flows from testing the gRPC response

for each endpoint's error cases. To test an entire flow, we need to be able to list the habits that we saved. Then, we'll write the first scenario.

10.5.1 List habits

Adding an endpoint requires a few additions, but it should be quick enough. Follow these steps:

1. Update the Protobuf file with a `ListHabits` endpoint, as shown in listing 10.23. This way, you can already publish the interface for the rest of your team to use and mock.

Listing 10.23 `service.proto`: Adding the list endpoint

```
// Habits is a service for registering and tracking habits.
service Habits {
  // CreateHabit is the endpoint that registers a habit.
  rpc CreateHabit(CreateHabitRequest) returns (CreateHabitResponse);

  // ListHabits is the endpoint that returns all habits.
  rpc ListHabits(ListHabitsRequest) returns (ListHabitsResponse);
}

// ListHabitsRequest is the request to list all the habits saved.
message ListHabitsRequest { #1
}

// ListHabitsResponse is the response with all the saved habits.
message ListHabitsResponse {
  repeated Habit habits = 1; #2
}
```

#1 This request has no parameter.

#2 This is how arrays are represented in Protobuf.

From there you can regenerate the corresponding Go files with the following:

```
> go generate ./...
```

2. Add the logic in the domain layer, following the pattern of `internal/habit/create.go`, as shown in listing 10.24. Don't forget to use a tiny interface for the

database, generate a mock for it, and test thoroughly. Note that it isn't always necessary to have a mock framework to test. An alternative to using a simple function replacing the database behavior is to have a structure holding the output content you want to mock and have the database call returning it.

Listing 10.24 list_test.go: Testing list without minimock

```
// MockList is a mock for FindAll method response.
type MockList struct { #1
    Items []habit.Habit
    Err    error
}

// FindAll is a mock which returns the passed list of items and error.
func (list MockList) FindAll(context.Context) ([]habit.Habit, error)
{
    return list.Items, list.Err
}

func TestListHabits(t *testing.T) {
    // TODO: Write the needed content for the tests cases

    "empty": {
        db:          MockList{
            Items: nil, Err: nil}, #2
        expectedErr: nil,
        expectedHabits: nil,
    },
    "2 items": {
        db:          MockList{Items: habits, Err: nil},
        expectedErr:  nil,
        expectedHabits: habits,
    },
    "error case": {
        db:          MockList{Items: nil, Err: dbErr},
        expectedErr:  dbErr,
        expectedHabits: nil,
    },
}

for name, tc := range tests {
    name, tc := name, tc

    t.Run(name, func(t *testing.T) {
        t.Parallel()

        got, err := habit.ListHabits(
            context.Background(), tc.db) #3
        assert.ErrorIs(t, err, tc.expectedErr)
        assert.ElementsMatch(t, tc.expectedHabits, got)
    })
}
}
```

#1 The database call output to mock: list of habits and the error

#2 Fills the expected output from the database call to the FindAll method

#3 Calls the mocked database interface

3. If you don't have one already, write the repository function that lists all the saved habits, as shown in the next listing. The repository should return a deterministic

list of habits, sorted using a specific criterion, such as the creation date of the habits.

Listing 10.25 `memory.go`: Deterministic output of habits

```
// FindAll returns all habits sorted by creation time.
func (hr *HabitRepository) FindAll(
    _ context.Context) ([]habit.Habit, error) {
    log.Infof("Listing habits, sorted by creation time...")

    // Lock the reading and the writing of the habits.
    hr.mutex.Lock()
    defer hr.mutex.Unlock()

    habits := make([]habit.Habit, 0)
    for _, h := range hr.habits {
        habits = append(habits, h)
    }

    // Ensure the output is deterministic by sorting the habits.
    sort.Slice(habits, func(i, j int) bool { #1
        return habits[i].CreationTime.Before(habits[j].CreationTime)
    })

    return habits, nil
}
```

#1 Inline implementation of sorting habits by creation time

4. Add the `ListHabits` method to the service, following the pattern of `internal/server/create.go`. Isolate the transformation of the generated structure into the domain structure in a separate function, and unit test it too. We decided that a repository containing no habits shouldn't be a problem and return an error, but feel free to make a different choice here. In addition, think about determinism: If the repository contains two elements, should they always be returned in the same order by the endpoint? Determinism is very important, and we can only recommend enforcing it wherever possible. Testing deterministic endpoints is a lot simpler than testing nondeterministic endpoints!

5. Test manually with `grpcurl` or a similar tool.

Now that you trust that this new endpoint works the way you expect, we can write an integration test.

10.5.2 Integration with go test

We want to write a test that will go through every layer of the service, all the way to the network outside of it. Considering that our database is currently a hacky in-memory thing, there's no point in mocking it, but when we finally use a real database system, it will be necessary to either mock it or run an instance locally. This test will run the service for real and call it as any client would.

RUN A SERVICE

First, create a test file as `internal/server/integration_test.go`, and add a `TestIntegration` function in the file, as shown in listing 10.26. There are several places an integration test file can be stored, but because we're testing the API of the server, we placed the file close to it.

First, we create a gRPC server instance and register it—something similar to what we already have in the `main` function. Second, we create a listener, and by giving an empty string as the address parameter, we ask it to find a free port on the host and use it. Third, we run that server in a parallel thread so that the rest of the test can keep running and calls can be made to it. Of course, we need the server to stop at the end of the test, whenever that is.

If we need to write any utility functions, we can start their implementation with `t.Helper()`. This will tell the Go test suite to ignore this layer when an error is surfaced.

Listing 10.26 integration_test.go: Starting the service

```
func TestIntegration(t *testing.T) {
    grpcServ := newServer(t)
    listener, err := net.Listen("tcp", "")
    require.NoError(t, err)

    wg := sync.WaitGroup{}
    wg.Add(1)
    go func() {
        defer wg.Done()
        err = grpcServ.Serve(listener)
        require.NoError(t, err)
    }()
    defer func() {
        // terminate the GRPC server
        grpcServ.Stop()
        // when that is done, and no error were caught, we can end this test
        wg.Wait() #1
    }()
}

func newServer(t *testing.T) *grpc.Server {
    t.Helper() #2
    s := server.New(repository.New(t), t) #3

    grpcServer := grpc.NewServer(
        grpc.UnaryInterceptor(timerInterceptor(s.lgr)))
    api.RegisterHabitsServer(grpcServer, s)
    reflection.Register(grpcServer)

    return grpcServer
}
```

#1 Using a WaitGroup ensures the goroutine has checked for any error returned by the Serve method.

#2 Skips this func when printing test details

#3 t has a Logf method, so it implements server.Logger and repository.Logger.

The server is running, so a client can enter the scene. We'll take care of that next.

CREATE A CLIENT

A client must know which address to send its requests to and the shape of the server (what the endpoints are). We create a function to build that new client, and it takes the address as a parameter, as shown in listing 10.27.

Note that we need to pass some credentials to connect to the server. The `grpc` library kindly offers a function that generates credentials that disable Transport Layer Security (TLS). While this is usually a security breach, we're running our server in a very restricted environment, and we can accept not having to pass credentials. Depending on which network the request will be sent through, you might have to use credentials, or you might be able to use the `insecure` package to generate some for you.

Listing 10.27 `integration_test.go`: Creating a client

```
func TestIntegration(t *testing.T) {  
    ...  
    // create client  
    habitsCli, err := newClient(t, listener.Addr().String())  
    require.NoError(t, err)  
}  
  
func newClient(t *testing.T, serverAddress string) (api.HabitsClient, error){  
    t.Helper()  
    creds := grpc.WithTransportCredentials(  
        insecure.NewCredentials()  
    ) #1  
    conn, err := grpc.Dial(serverAddress, creds)  
    if err != nil {  
        return nil, err  
    }  
  
    return api.NewHabitsClient(conn), nil  
}
```

#1 Insecure means “no TLS.”

The scene is set. Let's start the scenario.

RUN SCENARIO

As we're only testing two endpoints, we can create a function for the happy path for each of them. Then, we create an error path function for `CreateHabit` because `ListHabits` never returns a business error.

Listing 10.28 shows an example with the endpoint listing habits, which is the trickier of the pair, as it returns

generated IDs whose value will change at every run. So, we overwrite the `Id` field after checking that it has been filled.

Listing 10.28 `integration_test.go`: Function to list a Habit

```
func listHabitsMatches(
    t *testing.T, habitsCli api.HabitsClient, expected []*api.Habit) {
    list, err := habitsCli.ListHabits(
        context.Background(), &api.ListHabitsRequest{})
    require.NoError(t, err)

    for i := range list.Habits {
        assert.NotEqual(t, "", list.Habits[i].Id)
        list.Habits[i].Id = "" #1
    }
    assert.Equal(t, list.Habits, expected) #2
}
```

#1 This field's value was randomly generated. We only want to ensure that it's not empty.

#2 Use `ElementsMatch` for nondeterministic endpoints, and use `Equal` for deterministic ones.

Consider the option of basing your integration test on a struct that holds the client as a field and can hold methods that wrap calls to the client to make your test easier to read. We decided to use functions only, but all of them will start with the same two arguments, which can become very verbose—usually a cue for refactoring. With this kind of helper functions, the scenario can look fairly readable.

Listing 10.29 integration_test.go: Scenario in the code

```
// add 2 habits
addHabit(t, habitsCli, nil, "walk in the forest")
addHabit(t, habitsCli, ptr(3), "read a few pages")
addHabitWithError(t, habitsCli, 5, " ", codes.InvalidArgument)

// check that the 2 habits are present
listHabitsMatches(t, habitsCli, []*api.Habit{...})

// ...

func ptr(i int32) *int32 {
    return &i
}

func addHabit(t *testing.T, habitsCli api.HabitsClient,
    freq *int32, name string) {
    _, err := habitsCli.CreateHabit(context.Background(),
        &api.CreateHabitRequest{
            Name:        name,
            WeeklyFrequency: freq,
        })
    require.NoError(t, err)
}
```

Make sure you isolate your different scenarios so that they can run in parallel—you can even run that many instances of the server in parallel, one for each integration scenario. You can also use this opportunity to play with concurrency and call Add a large number of times concurrently to check for performance.

USING TEST.SHORT TO ONLY RUN LIGHTWEIGHT TESTS

So far, our test resembles any other unit test that we’ve written, except that it’s called an integration test. Sometimes, these integration tests can be quite intense because they go through lots of features and cases, they include some benchmarks, or they run some load or performance tests. These tests usually take quite some time, and it isn’t advised to include them in continuous integration (CI) toolchains, as they might slow down the delivery process. For instance, it could be a requirement to have “light” tests run on pull requests but “heavy” tests run

on tagging and image building. The `go test ./...` command accepts a `-short` flag. Setting this flag in the command line will change the output of the `testing.Short()` function, which we can invoke in any test. Let's start our `TestIntegration` with a check on that flag:

```
func TestIntegration(t *testing.T) {
    // Skip this test when running lightweight suites
    if testing.Short() {
        t.Skip()
    }

    grpcServ := newServer(t)
```

Now, let's take a look at the output of `go test -v ./...`. Using the `-v` flag makes the output verbose and lists each test function called. The output should contain `TestIntegration` as follows:

```
> go test -v ./...
...
ok      learn-go-pockets/internal/habit    1.004s
=== RUN TestIntegration
      create.go:17: Create request received: name="walk in the forest"
...
--- PASS: TestIntegration (0.00s)
...
```

Let's try the same, but this time with the short flag: `go test -v -short ./...`. This time, the output should explicitly indicate that `TestIntegration` was skipped:

```
> go test -v -short ./...
...
ok      learn-go-pockets/internal/habit    1.004s
=== RUN TestIntegration
      integration_test.go:23:
--- SKIP: TestIntegration (0.00s)
...
```

We now have a way of running only the lightweight tests in our CI pipelines. When running a more expensive or consuming one, check for that `-short` flag. Its presence should be an indicator that we want a quick result.

So far, we've been using our own in-memory database, which we trust. If that database were to be unavailable, we'd have serious problems in our server because the server and the database are part of the same program. But most of the time, the database is a remote entity, one that could behave erratically due to network problems, external load, or lots of other pesky bugs. Even worse, what if our own query crashes that database? We already handle the case when the database returns an error, but what if it doesn't answer our query? How long should we wait before realizing something is wrong?

10.6 Getting the best out of the context

When relying on remote services, a good practice is usually to allow the callee to provide a response within a specific time frame. There are two ways of expressing a time limit—either by providing a timeout, just like *Mission Impossible's* “This message will self-destruct in 5 seconds,” or by providing a deadline, “You have until Friday, 11 A.M.” The choice of which one to use depends on the activity being performed, but the former is more common than the latter when it comes to calls to remote network entities.

10.6.1 What is a context?

Earlier in the chapter, we mentioned but didn't define what a `Context` really is; it's now time to run `go doc context.Context` to find out. As we can read there, the purpose of a `Context` is to carry around deadlines, cancellation signals, and values across API boundaries. The documentation also tells us that a `Context` is an interface with the following methods:

- `Value(key any) any`

- `Deadline()` (deadline `time.Time`, ok `bool`)
- `Done()` `<-chan struct{}`
- `Err()` error

Before we go any further, it's important to note that even though `context.Context` is an interface, it's one of the few that you should never need to implement.

The `Value` method available on a `Context` is here to implement the “carrying values across API boundaries” requirement. While a `Context` can be seen as key-value storage, we highly recommend you don't think of it that way. If you place important values inside `Context`, then it's not clearly visible anymore what input different functions require. It's better to only stick to sending noncritical data, such as monitoring identifiers or requesting identifiers in the `Context`. Business values shouldn't be passed via the `Context`. If you're thinking of this as an option, we recommend going for an alternative. We'll see an example in section 10.6.3 of how to use the `Context`'s storage feature.

The `Deadline` method returns whether and when a `Context`'s deadline is set. The deadline is the time when the `Context` will start saying it's reached its expiration date to whomever might ask.

The `Done` method returns a channel that will be closed when the `Context` has reached its end. Calling `Done()` is simpler than comparing `Deadline()` with `time.Now()`, so this is usually done.

Finally, the `Err` method returns an error describing why the channel returned by `Done` is closed, or `nil` if it isn't closed yet. Now, let's take a look at how to create `Contexts`.

10.6.2 Create a context

Golang's `context` package offers several functions that allow for the creation of a `Context`. They are mostly divided into two categories: those that create a child `Context` from a parent one, and those that spring a `Context` out of the blue.

This latter set contains only `context.Background()` and `context.TODO()`. We recommend always creating your application's `Context` in your `main` function and then passing it around to any dependency that might need it. Create it with the `Background` method, and try to avoid calling `TODO`. Overall, your application should have a single parent `Context`.

Now that we've got a `Context`, we can create children. They can come in a variety of shapes, but the main difference lies in how we want to set their `Deadline` property. We can call `WithDeadline` to provide a specific timestamp at which time the child will be canceled, or `WithTimeout` if we want to specify how long a `Context` should "live." The second option is by far the most common when it comes to making calls to remote services.

Most of the time, however, a function will receive a `Context` as one of its parameters. There's a silent convention to always provide the `Context` as the first parameter of a call to a function—and, just as we usually name the errors `err`, we similarly very often call our `Contexts` `ctx`. When a function receives a `Context`, it shouldn't try to create a new one with `Background()` or `TODO()`. In our gRPC generated code, we can observe that the endpoints' signatures all start with an incoming `Context`—that's the one we should be using.

While it might seem repetitive to always have to pass the `Context` as a parameter, we strongly encourage you to resist the urge to use composition to save a `Context` into a variable.

10.6.3 Using a context

In which scenario should we create a child `Context`, though? Why not provide the parent `Context`? After all, it might have a deadline itself. The answer relates to good practices in coding and controlling with precision every call that is made across your network. If a service needs to call two other services to answer a request, we want to know which of these two is taking an awfully long time to return a response, if any. To achieve this, each remote call must be using its own `Context` with its own deadline. Depending on your application, a timeout value can range between 10 ms and some days. Don't be too strict, and be sure to take into account network latency.

Let's consider an example with our database and say we want to ensure that the repository call in our `Create` endpoint doesn't take too long. In listing 10.30, we show how to do this by creating a `Context` with a timeout of 100 ms, before we call our repository.

Listing 10.30 `create.go`: Adding a context around the db call

```
// Create adds a habit into the DB.
func Create(ctx context.Context, db habitCreator, h Habit) (Habit, error) {
    h, err := validateAndCompleteHabit(h)
    if err != nil {
        return Habit{}, err
    }

    dbCtx, cancel := context.WithTimeout(
        ctx, 100*time.Millisecond) #1
    defer cancel() #2
    err = db.Add(dbCtx, h) #3
    if err != nil {
        return Habit{}, fmt.Errorf("cannot save habit: %w", err)
    }

    return h, nil
}
```

#1 Always reuse the parent context.

#2 Makes sure we clean up after ourselves

#3 Don't forget to use the new context.

If we run this, everything works fine, but it's not tested, and we should test it. It's even worse than not tested—it actually breaks our existing tests! If you remember, we're using mocks, and mocks are very strict about what they expect. Our tests didn't worry too much about the `Context`—after all, we didn't fiddle with it so far. But now, the `Context` used to call `Add` in the `Create` endpoint isn't the `Background` one any more. We should update the test, but to provide the exact same `Context`, we need to know exactly when the deadline is to be able to expect it properly.

Many mocking libraries face this problem at some point. `Minimock` has decided to expose a `minimock.AnyContext` variable that will match any `context.Context` variable. Some other mocking libraries go a step beyond and propose a `mock.Anything` variable that can be used as a wildcard for any input parameter. We only need to use the `Context`, so we'll limit ourselves to this option.

Listing 10.31 `create_test.go`: Mocking the child context

```
func TestCreate(t *testing.T) {
    // ...
    "nominal": {
        db: func(ctl *minimock.Controller) *mocks.HabitCreatorMock {
            db := mocks.NewHabitCreatorMock(ctl)
            db.AddMock.Expect(minimock.AnyContext, h).#1
            Return(nil)
            return db
        },
        expectedErr: nil,
    },
    "error case": {
        db: func(ctl *minimock.Controller) *mocks.HabitCreatorMock {
            db := mocks.NewHabitCreatorMock(ctl)
            db.AddMock.Expect(minimock.AnyContext, h).#1
            Return(dbErr)
            return db
        },
        expectedErr: dbErr,
    },
}
```

#1 Replace the `ctx` with the `minimock` variable.

While this fixes the current tests, it doesn't test the current feature of having a timeout on our database call. For this,

we'll need to improve our mock.

As you now know, a `Context` is something that can reach its deadline, and, when this happens, the channel returned by `Done()` is closed, which means reading from this channel starts returning a zero value instead of being a blocking call. This is how applications check for a canceled/expired timeout. The following piece of code is present in various forms in most libraries that handle timeouts:

```
select {
// Read from channel used by backend to communicate response
case response := <-responseChan:
    return response, nil
// Check for deadline
case <-ctx.Done():
    return nil, ctx.Err()
}
```

In this `select`, whichever happens first causes the function to return—either we received a response, or the deadline was met. The line `case <- ctx.Done():` appears more than 60 times in the standard library alone, and it's always followed by returning the cause of the deadline via `ctx.Err()`.

Go 1.23 introduced functions that create a `Context` and that can cancel it with a “cause,” that is, a specific error. For instance, we now have a `context.WithTimeoutCause(parent Context, timeout time.Duration, cause error)` function. Calling `ctx.Err()` on these is quite ambiguous: Do we expect the `context.DeadlineExceeded` error to be returned, or do we expect the cause error we provided to `WithTimeoutCause`? To answer this, Go 1.23 provided a new function: `context.Cause(ctx context.Context)`. If no cause was set, this returns the usual error causing the `Context` to be canceled; however, if a cause was provided, that cause is returned by this function. Overall, we recommend using `context.Cause(ctx)` over `ctx.Err()`.

Let's add a test that implements this logic. For this, we can't use `Expect`, as this immediately returns the specified values. Instead, we'll have to overwrite the behavior of the `Add` method, which `minimock` allows with the `Set` method.

Listing 10.32 `create_test.go`: Testing the timeout

```
func TestCreate(t *testing.T) {
    // ...
    "db timeout": {
        db: func(ctl *minimock.Controller) *mocks.HabitCreatorMock {
            db := mocks.NewHabitCreatorMock(ctl)
            db.AddMock.Set(
                func(ctx context.Context, habit habit.Habit) error {
                    select {
                        // This tick is longer than a database call
                        case <-time.Tick(2 * time.Second):
                            return nil
                        case <-ctx.Done():
                            return ctx.Err()
                    }
                })
            return db
        },
        expectedErr: context.DeadlineExceeded, #1
    },
}
```

#1 This error is returned when a context reaches its deadline.

So, as we've seen, `Contexts` can be used to detect unexpectedly long remote calls. Some functions allow you to register key-value pairs inside a `Context`, but we recommend keeping that option as a last resort. The main reason is that a `context.Context` is only an interface that allows for the retrieval of values via a `Value` method, which means each network protocol might implement their own version of a `context.Context`. These versions will usually expose a way of adding a key-value pair into the `Context`. To achieve this purpose, the standard `context` package offers `subCtx := context.WithValue(ctx, key, value)`. Instead, let's resume our habits server, and implement the final endpoint—one that allows us to keep track of what we do on a weekly basis.

10.7 Track your habits

Congratulations, at this point of the chapter, you know all the basics about gRPC in Go! This next section is a guided exercise where you'll prove your autonomy and test your aggregated knowledge. If you struggle with anything here, you can find all the code in the repository (<https://mng.bz/EaBd>).

First, let's quickly reset the final goal of this pocket project and define what tracking a habit means. We can create a list of habits with a target of weekly frequency, tick a habit when we achieve it, and retrieve the current status so that we can plan the rest of the week. You'll be able to answer questions like these: Am I done for the week? Should I block a time slot to go for a walk? To do this, we'll build the following scenario:

1. Create several habits.
2. List the created habits.
3. Tick the habits that you achieve.
4. Get the status of the habits.

We're missing the bricks to fulfill steps 3 and 4, so let's go for the implementation. On the API, we'll need two new endpoints: `TickHabit` and `GetHabitStatus`.

10.7.1 Tick a habit

Let's define a new endpoint `TickHabit` on the Protobuf side with its associated request and response, as shown in the following listing. Don't forget to regenerate the Go library!

Listing 10.33 service.proto: TickHabit definition

```
// TickHabit is the endpoint to tick a habit.
rpc TickHabit(TickHabitRequest) returns (TickHabitResponse);
...

// TickHabitRequest holds the identifier of a habit to tick it.
message TickHabitRequest {
    // The identifier of the habit we want to tick.
    string habit_id = 1;
}

// TickHabitResponse is the response to TickHabit endpoint.
// Currently empty but open to grow.
message TickHabitResponse { #1
}
```

#1 Nothing yet, but having a message makes it easier to evolve

Then, add the implementation on the server with the following signature:

```
TickHabit(ctx context.Context, request *api.TickHabitRequest)
(*api.TickHabitResponse, error)
```

Because ticks and habits are different concepts, we store them in different tables in a SQL database. In our memory implementation, we'll store the ticks in a structure of their own, next to the habits. If we develop a UI, this will allow us to retrieve only the list of habits or the full status of a habit for a week.

10.7.2 Store ticks per week

This is the same logic that we used previously for the habits in memory; the only tricky part is the data definition. We want to store all the ticks for each habit grouped by week, so we can get a weekly status. The built-in Go library package provides a very useful method named `ISOWeek()` that returns the ISO 8601 year and week number in which that time occurs. Running `go doc time.ISOWeek` returns the following:

```
func (t Time) ISOWeek() (year, week int)
```

We'll use ISOWeek in our code. Let's create a new package called `isoweek` and a new file where we'll define an `ISO8601` structure holding a Year and a Week.

Listing 10.34 `isoweek/isoweek.go`: `ISO8681` structure

```
package isoweek

// ISO8601 holds the number of the week and the year.
type ISO8601 struct {
    Year int
    Week int
}
```

It's now easy to define the data storage type in `Repository`. To retrieve the status of a habit for the current week, our storage will store a map for each habit ID of the `isoweek` with its associated events, which are timestamps:

```
storage map[habit.ID]map[isoweek.ISO8681][]time.Time
```

For more readability, we chose to have a custom type called `ticksPerWeek`, which is a map holding all the timestamps per ISO week. It will now be very easy to retrieve the status of a habit at the current time. If you want to extend the project, you can even have an endpoint retrieve the status for a given week or date. Let's add a new type of storage to `HabitRepository` and rename `db` into `habits` to be more explicit.

Listing 10.35 `repository/memory.go`: `ticks` storage

```
// ticksPerWeek holds all the timestamps for a given week number.
type ticksPerWeek map[isoweek.ISO8601][]time.Time

// HabitRepository holds all the current habits.
type HabitRepository struct {
    habits map[habit.ID]habit.Habit #1
    ticks  map[habit.ID]ticksPerWeek #2
}
```

#1 Refactors `db` field into `habits`

#2 Adds a new storage for ticks

Do you feel confident in creating the needed methods? Let's not anticipate and create only the `Add` for the moment. All

the logic of ISO8601 computation is done in a dedicated function, so if we pass it directly as a parameter, it will be easy to reuse over the other endpoints and tests:

```
AddTick(_ context.Context, id habit.ID, t time.Time) error
```

Don't forget to verify whether the habit and the ISOWeek exist in the storage before inserting a new tick. As shown in the following listing, the full implementation of `Tick` on the domain side should now be ready!

Listing 10.36 habit/tick.go: Tick implementation

```
package habit

import (
    "context"
    "fmt"
    "time"
)

//go:generate minimock -i habitFinder -s "_mock.go" -o "mocks"
type habitFinder interface {
    Find(ctx context.Context, id ID) (Habit, error)
}

//go:generate minimock -i tickAdder -s "_mock.go" -o "mocks"
type tickAdder interface {
    AddTick(ctx context.Context, id ID, t time.Time) error
}

// Tick inserts a new tick for a habit.
func Tick(ctx context.Context, habitDB habitFinder, tickDB tickAdder, id ID,
    t time.Time) error {
    // Check if the habit exists.
    _, err := habitDB.Find(ctx, id)
    if err != nil {
        return fmt.Errorf("cannot find habit %q: %w", id, err)
    }

    // AddTick adds a new tick for the habit.
    err = tickDB.AddTick(ctx, id, t)
    if err != nil {
        return fmt.Errorf("cannot insert tick for habit %q: %w", id, err)
    }

    return nil
}
```

#1 Checks that the habit exists in the repository

We now just have to call it on the server side and transform the request and the response. Because we need to provide a timestamp when the habit was ticked, we could have it passed by either the caller or the gRPC endpoint. If the value is set in the server layer rather than read from the request, we need to remember that the server and the client might be in different time zones and that the “current day” is only a relative notion. Wait! What happens if someone tries to tick a habit that doesn’t exist?

10.7.3 Handle corner cases

If the habit doesn’t exist in the habits repository, we don’t want to have inconsistent data and store a new tick for an unknown habit. Let’s create a `Find` method on the habit repository:

```
Find(ctx context.Context, id habit.ID) (habit.Habit, error)
```

A good practice is to create a custom error that is checked on the server side to return the proper gRPC code. Here, we chose to switch on the domain error and convert in `codes.NotFound`, for example, if the habit doesn’t exist in the database:

```
err := habit.Tick(ctx, s.db, s.db, habit.ID(request.HabitId), time.Now())
if err != nil {
    switch {
    case errors.Is(err, r.ErrNotFound):
        return nil, status.Errorf(codes.NotFound,
            "couldn't find habit %q in repository", request.HabitId)
    default:
        return nil, status.Errorf(codes.Internal,
            "cannot tick habit %q: %s", request.HabitId, err.Error())
    }
}
```

You can test the endpoint manually using `grpcurl` on a created habit, for instance, one with an ID of `98ab1bbe-41d5-4ed3-8f33-e4f7bec448c8`:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"habit_id":"98ab1bbe-41d5-4ed3-8f33-e4f7bec448c8"}' \
localhost:28710 \
habits.Habits/TickHabit
```

Upon inspection, no errors have been returned. Our interest now lies in determining the frequency of calls made to the tick endpoint for a given habit. Let's retrieve this information next.

10.7.4 Get habit status

The last task entails retrieving the count of habit ticks for a given week. This requires both the habit ID and a timestamp to specify the desired week. We'll implement an endpoint capable of accepting the ID and timestamp parameters, which will then furnish habit details alongside the tick count. The Potobuf definition looks like the following listing.

Listing 10.37 api/proto/service.go: GetHabitStatus definition

```
// GetHabitStatus is the endpoint to retrieve the status of
// ticks of a habit.
rpc GetHabitStatus(
    GetHabitStatusRequest
) returns (GetHabitStatusResponse);

// GetHabitStatusRequest is the request to GetHabitStatus endpoint.
message GetHabitStatusRequest {
    // The identifier of the habit we want to retrieve.
    string habit_id = 1;

    // The time for which we want to retrieve the status of a habit.
    optional google.protobuf.Timestamp timestamp = 2;
}

// GetHabitStatusResponse is the response to retrieving the status of a
// habit.
message GetHabitStatusResponse {
    // All the information of a habit.
    Habit habit = 1;
    // The number of times the habit has been ticked for a given week.
    int32 ticks_count = 2;
}
```

At this point, you should be well equipped to do the remainder of the steps:

1. Create a method on the server side.
2. Isolate the logic on the domain.
3. Retrieve the data on the repository side.
4. Plug all the calls.
5. Don't forget to test!

If you test with `grpcurl`, you should get something like this:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"habit_id":"98ab1bbe-41d5-4ed3-8f33-e4f7bec448c8"}' \
localhost:28710 \
habits.Habits/GetHabitStatus

{
  "habit": {
    "id": "98ab1bbe-41d5-4ed3-8f33-e4f7bec448c8",
    "name": "read a few pages",
    "weeklyFrequency": 3
  },
  "ticksCount": 2
}
```

10.7.5 Add a timestamp

Until we pick up the habit of remembering to tick the habits we complete, it would be helpful to have the option to dive into the past to tick a habit we forgot to update. To do so, we can extend the two last endpoints by adding a timestamp to the requests.

In Protobuf, you can import different types that will be nicely serialized in the programming language you choose. You can always refer to the full list of well-known types in the Protocol Buffers documentation (<https://mng.bz/avXz>). The expected format is an RFC3339 date string such as 2024-01-25T10:05:08+00:00. Let's import the timestamp type by adding this line in the top imports of our `service.proto` file:

```
import "google/protobuf/timestamp.proto";
```

The following listing shows an example of how to add the timestamp as a new field `GetHabitStatusRequest`, using the Protobuf `Timestamp` type. By default, a field is optional in Protobuf version 3 (meaning a message without that field will be considered valid), and the usage of the `required` keyword has been discontinued (meaning a message without that field will be considered invalid).

Listing 10.38 `api/proto/service.go`: Importing the timestamp

```
message GetHabitStatusRequest {  
    string habit_id = 1;  
    google.protobuf.Timestamp time = 2;  
}
```

Sometimes, dealing with time as a parameter is a bit tricky—the server and the client might be in different time zones, which makes 8:30 A.M. unclear, for example. We recommend always using the server as the truth when asking what time it is. For now, we’ll keep this field here, as it will be useful for tests. Update the two endpoints, and test your code!

10.7.6 Habit tracker in action

We’re now able to play with the habit tracker, so let’s play a full scenario where we add habits, tick them, retrieve their status, tick them again with a timestamp, and retrieve their status for the given date. You can do it manually by using `grpcurl` in your terminal, or you can update the integration test. Let’s first see the `grpcurl` commands and compare the responses:

1. Create this habit: “Write some Go code.”
Request:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"name":"Write some Go code", "weekly_frequency":3}' \
localhost:28710 \
habits.Habits/CreateHabit
```

Response:

```
{
  "habit": {
    "id": "94c573f1-df03-45ec-97fc-8b8fc9943472",
    "name": "Write some Go code",
    "weeklyFrequency": 3
  }
}
```

2. Create this habit: “Read a few pages.”

Request:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"name":"Read a few pages", "weekly_frequency":5}' \
localhost:28710 \
habits.Habits/CreateHabit
```

Response:

```
{
  "habit": {
    "id": "96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
    "name": "Read a few pages",
    "weeklyFrequency": 5
  }
}
```

3. Retrieve the list of habits.

Request:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{} ' \
localhost:28710 \
habits.Habits/ListHabits
```

Response:

```
{
  "habits": [
    {
      "id": "94c573f1-df03-45ec-97fc-8b8fc9943472",
      "name": "Write some Go code",
      "weeklyFrequency": 3
    },
    {
      "id": "96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
      "name": "Read a few pages",
      "weeklyFrequency": 5
    }
  ]
}
```

4. Tick the “Write some Go code” habit without a timestamp because you just did it.

Request:

```
> grpcurl \
  -import-path api/proto/ \
  -proto service.proto \
  -plaintext -d '{"habit_id":"94c573f1-df03-45ec-97fc-8b8fc9943472"}' \
  localhost:28710 \
  habits.Habits/TickHabit
```

Response:

```
{
}
```

5. Get the status of the “Write some Go code” habit for the current week.

Request:

```
> grpcurl \
  -import-path api/proto/ \
  -proto service.proto \
  -plaintext -d '{"habit_id":"94c573f1-df03-45ec-97fc-8b8fc9943472"}' \
  localhost:28710 \
  habits.Habits/GetHabitStatus
```

Response:

```
{
  "habit": {
    "id": "94c573f1-df03-45ec-97fc-8b8fc9943472",
    "name": "Write some Go code",
    "weeklyFrequency": 3
  },
  "ticksCount": 1
}
```

6. Tick the “Read a few pages” habit without a timestamp because you’re doing it.

Request:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"habit_id":"96b72dce-7a2e-43ce-9091-0f9fc447b8a1"}' \
localhost:28710 \
habits.Habits/TickHabit
```

Response:

```
{
}
```

7. Get the status of the “Read a few pages” habit for the current week.

Request:

```
> grpcurl \
-import-path api/proto/ \
-proto service.proto \
-plaintext -d '{"habit_id":"96b72dce-7a2e-43ce-9091-0f9fc447b8a1"}' \
localhost:28710 \
habits.Habits/GetHabitStatus
```

Response:

```
{
  "habit": {
    "id": "96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
    "name": "Read a few pages",
    "weeklyFrequency": 5
  },
  "ticksCount": 1
}
```

8. Tick the “Read a few pages” habit with a timestamp in the previous week.

Request:

```
grpcurl \
  -import-path api/proto/ \
  -proto service.proto \
  -plaintext -d '{"habit_id":"96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
  -"timestamp": "2024-01-24T20:24:06+00:00"}' \
  localhost:28710 \
  habits.Habits/TickHabit
```

Response:

```
{
}
```

9. Get the status of the “Read a few pages” habit during the previous week.

Request:

```
grpcurl \
  -import-path api/proto/ \
  -proto service.proto \
  -plaintext -d '{"habit_id":"96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
  -"timestamp": "2024-01-24T20:24:06+00:00"}' \
  localhost:28710 \
  habits.Habits/GetHabitStatus
```

Response:

```
{
  "habit": {
    "id": "96b72dce-7a2e-43ce-9091-0f9fc447b8a1",
    "name": "Read a few pages",
    "weeklyFrequency": 5
  },
  "ticksCount": 1
}
```

Launching the commands manually can be a bit annoying and repetitive, so let’s automate this and update the integration test, as shown in listing 10.39. First, we can write helper functions as we did previously to tick a habit and to verify that the habit status matches when calling

GetHabitStatus. These functions will both make the call to the API and validate the returned values.

Listing 10.39 integration_test.go: TickHabit and GetStatus calls

```
func tickHabit(t *testing.T, habitsCli api.HabitsClient, id string) {
    _, err := habitsCli.TickHabit(context.Background(), &api.TickHabitRequest{
        HabitId: id,
    })
    require.NoError(t, err) #1
}

func getHabitStatusMatches(t *testing.T, habitsCli api.HabitsClient,
    id string, expected *api.GetHabitStatusResponse) {
    h, err := habitsCli.GetHabitStatus(context.Background(),
        &api.GetHabitStatusRequest{HabitId: id})
    require.NoError(t, err)

    assert.Equal(t, expected.Habit, h.Habit) #2
    assert.Equal(t, expected.TicksCount, h.TicksCount)
}
```

#1 Fails if TickHabit returns an error

#2 Verifies the content of the GetHabitStatus response

The generated ID is needed to call TickHabit and GetHabitStatus endpoints. We can retrieve the ID from the addHabit helper method, as shown in the following listing.

Listing 10.40 integration_test.go: Updating addHabit to retrieve the ID

```
func addHabit(t *testing.T, habitsCli api.HabitsClient,
    freq *int32,
    name string) string { #1
    resp, err := habitsCli.CreateHabit(context.Background(),
        &api.CreateHabitRequest{
            Name:      name,
            WeeklyFrequency: freq,
        }) #2
    require.NoError(t, err)

    return resp.Habit.Id #3
}
```

#1 We need to update the signature to return the ID and to update the main test function.

#2 Retrieves the response from the endpoint

#3 Returns the habit ID

With the following listing, you can now add the calls to the main test function by calling the two helpers tickHabit and

getHabitStatusMatches, shown earlier in listing 10.39.

Listing 10.41 integration_test.go: Calling tickHabit/verify statuses

```
func addHabit(t *testing.T, habitsCli api.HabitsClient, freq *int32,
    name string) string {
    // add 2 ticks for Walk habit
    tickHabit(t, habitsCli, idWalk)
    tickHabit(t, habitsCli, idWalk)

    // add 1 tick for Read habit
    tickHabit(t, habitsCli, idRead)

    // check that the right number of ticks are present
    getHabitStatusMatches(t, habitsCli, idWalk, &api.GetHabitStatusResponse{
        Habit: &api.Habit{
            Id:          idWalk,
            Name:        "walk in the forest",
            WeeklyFrequency: 1,
        },
        TicksCount: 2,
    })

    getHabitStatusMatches(t, habitsCli, idRead, &api.GetHabitStatusResponse{
        Habit: &api.Habit{
            Id:          idRead,
            Name:        "read a few pages",
            WeeklyFrequency: 3,
        },
        TicksCount: 1,
    })
    ...
}
```

Congratulations! You've built a solid habit tracker backend that can be reused easily for a frontend application. You can commit and enjoy your new project!

Summary

- The `go generate` command is a Go tool that enables you to generate programs from the source code. The compiler scans comments with the specific syntax `//go:generate` and executes the command that follows on the rest of the line.
- gRPC is a framework to connect services, devices, applications, and more. It's a powerful and efficient

framework for transporting lightweight messages in Protobuf format.

- The Protocol Buffers framework (Protobuf) provides serialization for structured data while guaranteeing high performance. It comes with its own syntax composed of Protobuf messages and services written in .proto files.
- Protobuf messages are language neutral. Thanks to the Protobuf compiler (`protoc`), you can generate interfaces and structures in many programming languages (Go, Java, C++, etc.).
- Communication between gRPC clients and servers is standardized thanks to status codes defined by the RPC API. A status is composed of an integer code and a string message. While designing the API, you should pick the most appropriate return code for your use case (you can always refer to the documentation at <https://mng.bz/gapl>).
- `grpcurl` is an open source command-line interface (CLI) tool that enables you to communicate with gRPC servers easily. It's basically like `curl` but for gRPC. This human-friendly CLI allows you to define JSON requests instead of unreadable bytes. It's very useful when you need to test services manually.
- Declare small interfaces close to their use. It comes in handy when testing to mock your dependencies instead of counting on hardcoded behaviors.
- Dependency injection is a technique to give all the needed objects to a function instead of creating or building them internally. Note that it's useful to mock the dependencies when testing the function.
- While testing, there are different ways of simulating dependency behaviors, and mock tools are great for describing expected results. The main well-known mock

tools are `mockgen` (comes with Go), `mockify`, and `minimock`, which we used in this chapter.

- The standard `context` library provides the `Context` type and its associated methods to carry information along requests and responses. For example, when a user sends a request, you can store its identity in the `Context` and retrieve it later in the chain of functions. It's safe to use methods such as `WithCancel` or `WithDeadline` across multiple goroutines.
- A program should create only one `Context` and provide it to its functions. Most of the time, the `Context` will be coming from an external caller. Creating children for a `Context` is perfectly fine and encouraged.
- Always pass a `context.Context` as the first parameter of a function, even if you're not using it (you can use the blank identifier in this rare case).
- A `Context` is necessary any time we make a remote call across the network, whichever protocol or framework is being used. Setting a deadline for the remote call is a good safety net; otherwise, your application calls might be hanging forever.
- To check if a `Context` has expired, try reading from `<- ctx.Done()`. This will return something when the `Context` is no longer usable. To know why a `Context` is over, check its error with `ctx.Err()` or, even better, with `context.Cause(ctx)`.
- Don't use a `context.Context` as a key-value storage inside an application.
- Mocking functions that use a `Context` is sometimes tricky, especially if the function being tested creates a `context` of its own. To solve this, many libraries offering mocks expose a variable that can be used as a wildcard for the `Context`.

- Generated mocks are programmed to behave as specified. If they receive a call for a precise set of parameters, they will return the specified values. However, it's sometimes necessary to override this default behavior of always returning something, especially when testing the behavior when a deadline is reached.
- Using `testing.Short()` allows us to know if the `-short` flag was passed on the `go test` command line. Long or expensive tests should be skipped altogether when this flag is set, by using `t.Skip()`.
- Dealing with “current time” is tricky because asking what time it is provides different results based on the time zone of the person being asked. As a result, we recommend checking for current time on the server side, rather than expecting the client to provide that value.

11 HTML templating with a gRPC client

This chapter covers

- Creating a gRPC client
- Using templates in Go to fill an HTML page
- Calling an existing backend

When you have data that you want to show, there are two main questions to answer: What should you show, and how should you show it? While answering the first question is part of understanding the problem, knowing how to present the results, and making them clear, is often a requirement to make the solution meaningful. As computer scientists, we deal with huge amounts of data, most of which is mundane to the people who live next door, the other school parents, or the person in front of us at the grocery store. In chapter 5, we explained how using icons made the game Wordle (and our game Gordle) more entertaining than a display of mere numbers. Being able to clearly give a picture of the raw data is so important that the Golang developers introduced the `text/template` package to support this in the first version of the language. The goal of this package and of its sibling `html/template` is to generate enriched text output based on the contents of some data. Most of the time, the data needs to be nicely formatted—add a tab here, insert a new line there, or add this or that tag—and we'll explain why other Golang tools aren't better at this task than templates.

Templating has been intensively used by tools such as Helm to factor YAML deployment configuration files and make them more generic. In this chapter, we'll use Go's template packages to provide a nice UI on top of our habits tracker that we wrote in the previous chapter. For this, we'll interact with a web server that displays an HTML page containing all the important information about our habits. Let's face it—`grpcurl` is useful, but having buttons will be a much better user experience—even if it's only for tests! This project has the following requirements:

- The user must be able to access an HTTP server via a web browser.
- The web page exposes the current habits in a visually pleasing way.
- The web page allows the user to tick a habit for this week and to create new habits.
- The HTTP server communicates with our gRPC habits service.
- The HTTP server returns an HTML document containing the habits.

Note This chapter relies on the code from chapter 10. If you skipped it, you can use the code from the book's repository.

11.1 Your basic HTTP server

Let's begin by examining figure 11.1 to gain a comprehensive understanding of our objectives for this chapter. Currently, we've implemented the gRPC habits service and the storage logic. Next, we need an HTTP server to convert our content into an esthetically pleasing HTML page, which will be served to our users' web browsers.

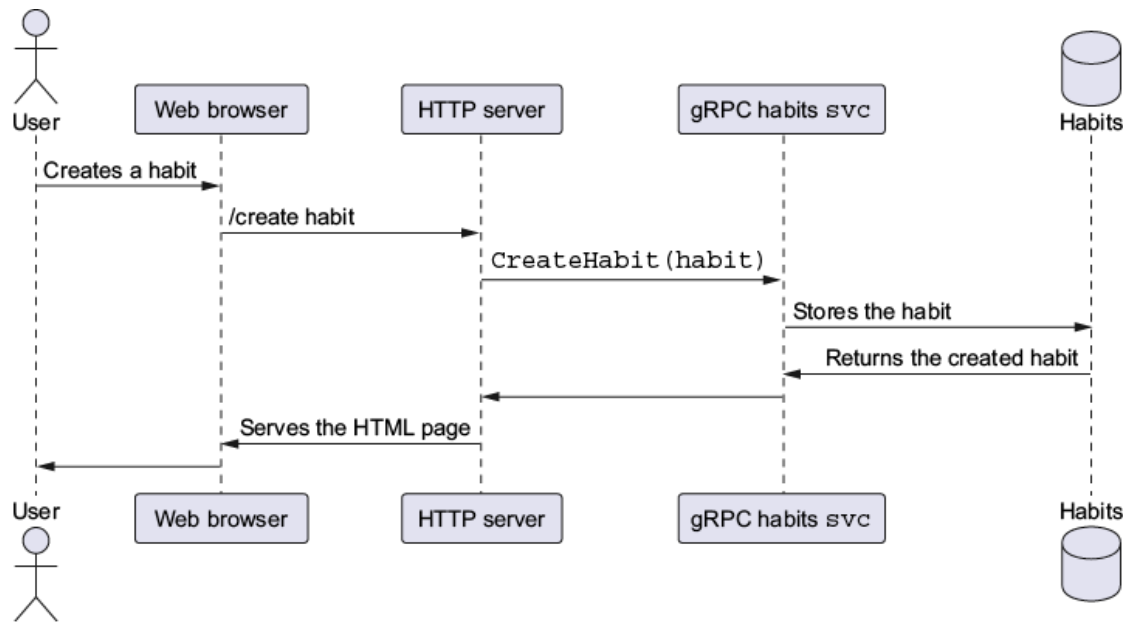


Figure 11.1 A unified modeling language diagram of the global architecture

In this first part, we want to run a server that returns some HTML to a web browser. First, let's run a server that returns something, just to check that it works. This is like a `hello, world` on HTTP, at the root path of the service, namely, `localhost:8083/`. Pick the port you like—usually 8080 and above is a good range during development. This server will be the bridge, receiving HTTP requests from the web page the user is browsing, and sending adequate Protobuf messages to the gRPC server on the other side.

In chapter 8, you saw the basics of an HTTP server, but let's recap. Create your module in a new folder, run `go mod init learn-go-pockets/templates` or any other module name, and create a `main.go` in the `cmd/httpserver` folder. Writing this server can be done in a handful of lines.

Listing 11.1 Minimal HTTP server

```
func main() {  
    http.HandleFunc("/",  
        func(w http.ResponseWriter, r *http.Request) { #1  
            io.WriteString(w, "ሰላም፣ሰዎች።") #2  
        })  
  
    err := http.ListenAndServe(":8083", nil) #3  
    if err != nil {  
        fmt.Fprintln(os.Stderr, "Error while serving: " + err.Error())  
    }  
}
```

#1 Defines a handler function for the root ("/") path

#2 The contents of the page

#3 Starts the HTTP server on port 8083

Run it, open a browser, and visit `localhost:8083`. When displaying this page, a request is sent by the browser to local port 8083. There, it's received by our server, which sends the following response: "ሰላም፣ሰዎች።" (Amharic for “Hello, world.”—and, no, the comma and period aren't the “,” and “.” characters here). This request is then received and displayed by the browser. You can go run `cmd/httpserver/main.go` and open a browser to `http://localhost:8083`. Do you see the message? Let's make this better for future maintainers.

11.1.1 Hello plain text

Of course, this flat code structure won't take us very far, so we need to organize the code before it grows. Following our choices from chapter 8, we can create an `internal/handlers` package, where we'll put the logic of what happens when a path is requested. Traditionally, in plain HTML, the default page to show is called the index, so let's keep this naming. Create a file for the server structure and a file for the index logic:

```
> tree
.
├── cmd
│   └── httpserver
│       └── main.go
├── go.mod
├── go.sum
└── internal
    └── handlers
        ├── index.go
        └── server.go
```

MAIN FUNCTION

First, let's update the `main` function to create a new server, listen, and serve requests using the router, as shown in listing 11.2. We'll implement the server next. As you notice, listening to a port and serving doesn't change from our previous straightforward implementation, except for the router parameter: we want all the routes to be managed by the server that we'll write next.

Listing 11.2 `main.go`: Using the server's router

```
const port = 8083

func main() {

    srv := handlers.New() #1

    addr := fmt.Sprintf(":%d", port)
    // TODO: Log a message: we are listening

    err := http.ListenAndServe(addr, srv.Router()) #2
    if err != nil {
        panic(err)
    }
}
```

#1 Creates the server but doesn't compile

#2 Listens on port 8083

Unhappy compilers make unhappy developers, so let's implement this server.

SERVER STRUCTURE

As usual, we want the server to be a struct holding its dependencies, so we can put it under test with the relevant mocks. Before we add the gRPC client as a dependency, the first function that this structure needs must build a router, linking to its different endpoints. We saw in chapter 8 why we picked the standard library as our routing library, rather than `go-chi`, `fiber`, or `gorilla`. Let's do it again here. Write the `Server` struct with this one field, and add a `New()` function to build it, taking no parameters yet.

Listing 11.3 `server.go`: Server struct

```
// Server serves all the HTML routes on this service.
type Server struct {} #1

func New() *Server... #2

// Router returns an HTTP handler that listens to all the proper paths.
func (s *Server) Router() http.Handler {
    r := http.NewServeMux() #3

    // Register each endpoint.
    r.HandleFunc(http.MethodGet+" "+"/", s.index) #4

    return r
}
```

#1 Defines an empty struct

#2 Implements New

#3 Creates the server

#4 Registers the handler “s.index” to the route “/” for the verb GET

Each page, endpoint, or path will be processed by a method on our `Server` structure. Now we need to implement the `index` method, which tackles the root of the server.

INDEX PAGE

What logic will run whenever a client navigates to the index page? From the preceding code listing, our server knows

that it must call the `index` method, which needs to return a status code and some page content.

As you can see in the documentation, the `HandleFunc` function declares an HTTP route with the eponymous verb and directs the request to the second parameter, which is an `http.HandlerFunc`. This means we need to create the method `index` with a specific signature, defined in the documentation as follows:

```
// The HandlerFunc type is an adapter to allow the use of
// ordinary functions as HTTP handlers.
// [...]
type HandlerFunc func(ResponseWriter, *Request)
```

Write the `index` method with this signature in a dedicated file, for example, `index.go`:

```
// index serves the root page of the app.
func (*Server) index(w http.ResponseWriter, r *http.Request) {}
```

The only operation that this function needs to do is to output some HTML on the `w` variable. Should we use `fmt.Fprint` or `io.WriteString`? When writing a string to a writer, two main options are available: `io.WriteString(w, ...)` or the family of `fmt.Fprint(w, ...)`. If what needs to be printed is a hardcoded string, `WriteString` is perfect. However, as soon as you need to add information to the string and make it more dynamic, using the `fmt` package becomes a necessity. Now that this is cleared up, let's output our Hello, World again and see if it works after refactoring.

Listing 11.4 `index.go`: Handler for the index page

```
// index serves the root page of the app.
func (_ *Server) index(w http.ResponseWriter, _ *http.Request) {
    io.WriteString(w, "Hello, World!")
}
```

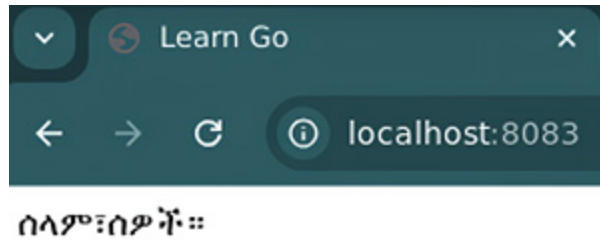


Figure 11.2 A first page—not yet HTML

Run the server, refresh the page at `localhost:8083`, and see what happens. Figure 11.2 shows the first page.

It's cute, but we're boldly ignoring a potential error returned by `WriteString`! If an error occurs, the first thing to do is log it. Using `fmt.Println` will help in development but not in production or during tests where the logs will get mangled in the test output. Let's delay the answer to this till the end of this section.

Note When there are errors writing the HTML response, as soon as we call `Write` using the response writer, the HTML headers are set, and we can't change them anymore by using `Error`. The only remaining option is to log the error.

The log is for developers who will have to debug the problem, but we also need to tell the client what happened:

```
, err := io.WriteString(w, "Hello, World!")
if err != nil {
    // TODO: Log a message.
    fmt.Println("Error while rendering: " + err.Error())
}
```

With this, the user's browser can know how to interpret the response.

RESPONSEWRITER'S DEFAULT BEHAVIOR

Why aren't we explicitly returning a status when everything is correct? If you look at the `WriteString` documentation, it states that in our case, "[`Writer.Write`] is called exactly once." Click along in the documentation, and you'll find out that `http.ResponseWriter` has a lot of useful behaviors to make your life easier: "If [`ResponseWriter.WriteHeader`] hasn't yet been called, `Write` calls `WriteHeader(http.StatusOK)` before writing the data. Additionally it deals with the `Content-Type` and `Content-Length` headers in the majority of situations." Of course, we always get over the edge of that "majority" case when least expected, so keep it in mind when your hotfix suddenly breaks everything.

HTML BASICS

This book isn't about frontend technologies, and none of these authors would dare claim any expertise in the area. What we're good at, though, is reading documentation (and looking up things on the internet). The W3C, responsible for the web's norms, states the following:

- All HTML documents must start with a document type declaration: `<!DOCTYPE html>`.
- The HTML document itself begins with `<html>` and ends with `</html>`.
- The visible part of the HTML document is between `<body>` and `</body>`.
- The HTML `<head>` element is a container for the following elements: `<title>`, `<style>`, `<meta>`, `<link>`, `<script>`, and `<base>`.

We can replace the `hello, world` with some HTML code.

Listing 11.5 index.go: HTML for the index page

```
_, err := io.WriteString(w, `#1
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Learn Go</title> #2
</head>
<body>
  <h1>YAY</h1> #3
  <p>Way to go!</p> #4
</body>
</html>
`)
```

#1 Uses backquotes to escape double quotes

#2 A title for the browser's tab

#3 The main title of the page

#4 Message to display

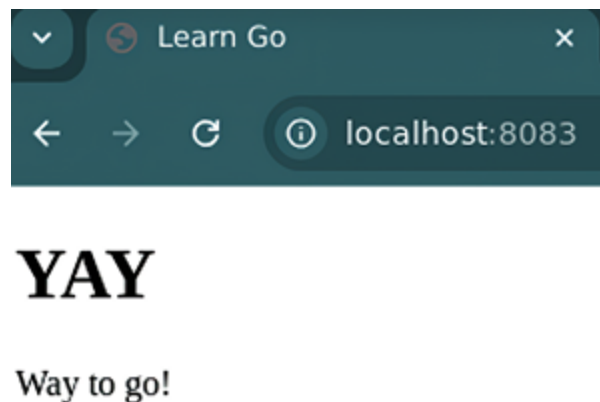


Figure 11.3 The page in a browser

Run it and enjoy the experience of the first days of the web, when people were writing HTML by hand. Figure 11.3 shows the page as it will appear in a browser.

Looking good, but we still have a TODO that needs some attention.

USE A PROPER LOGGER

We now need to add a proper logger to the server. Let's reuse a logger similar to what we used in chapter 10: a type `Logger` in charge of writing to a specified output. This type

only exposes one method, `Logf`. This logger will be a dependency of the server, where we'll declare it as an interface to help with tests.

Listing 11.6 `log.go`: Implementing a lightweight logger

```
// Logger is a lightweight logger.
type Logger struct {
    lgr *log.Logger #1
}

// New returns a logger that prints to the specified output.
func New(w io.Writer) *Logger {
    return &Logger{lgr: log.New(
        w, "", log.Ldate|log.Ltime)} #2
}

// Logf formats the message and sends it to the output.
func (l *Logger) Logf(format string, args ...any) {
    l.lgr.Printf(format, args...)
}
```

#1 Our logger uses the `log` package internally.

#2 Adds some default formatting parameters

Now, we can add a logger dependency in our server. For this, we don't want to rely on the `log.Logger` type, as this will make testing our code painful. When we test the `server` package, we don't also want to be affected by code in the `log` package. Instead, we can declare the local interface that we expect the logger to implement.

TIP “Accept interfaces, return types” is a little axiom that should help you design the signatures for your functions and methods. You can always define a minimal interface for a parameter. For the return type, there's usually no reason to return an interface, unless you specifically want to return something generic.

Our choice, in the `log/logger.go` file, was driven by what the `testing.T` structure already implements—a method called `Logf`, as shown in the following listing. This will allow us to use a `testing.T` in our tests instead of creating a `Logger` variable.

Listing 11.7 server.go: Receiving a logger

```
type Logger interface { #1
    Logf(format string, args ...any)
}

type Server struct {
    lgr    Logger #2
}

func New(lgr Logger) *Server { #3
    return &Server{lgr: lgr}
}
```

#1 Defines a local interface

#2 Stores the logger in the server struct

#3 Receives an injected logger

Finally, all we have left to do is update the `main` function, where we can now build the logger, and inject it into our server when creating it. We'll now be able to log what's going on in the backend when we tick a habit or perform any other action. The server's `os.Stdout` won't be sent to the client and will remain "internally" on the server's premises for developers to investigate.

UNIT TESTING HTML

For the moment, the `index` outputs a static HTML document. We want to manage our future habits with a nice UI. To make sure it stays nice, let's set up testing using the `httptest` package.

First, we check that everything went well, and we get a 200 OK response when we call `index`. We can use the popular github.com/stretchr/testify/assert and github.com/stretchr/testify/require packages here.

As we saw in chapter 8, we need a response recorder from the `httptest` package to test the output of an HTTP request.

Listing 11.8 `index_test.go`: Checking the status code

```
func TestServer_index(t *testing.T) {  
    rr := httptest.NewRecorder() #1  
  
    s := New(t) #2  
    s.index(rr, nil) #3  
  
    assert.Equal(t, http.StatusOK,  
        rr.Result().StatusCode) #4  
}
```

#1 Creates a response recorder

#2 The `testing.T` structure implements our `Logger` interface.

#3 Calls the handler for the path `"/`

#4 Checks the status code

To check the contents of the response, we can have a real HTML file with the expected output as our reference. It's a test fixture, so it should go in a `testdata` folder:

```
expect, err := os.ReadFile("testdata/index.xhtml")
```

In tests, such files are referred to as *golden files*. They contain the absolute truth of what the generated contents should be. It's sometimes customary to give them a `.golden` suffix.

If you use a simple equality between the two strings, you may be surprised by differences caused by the tabs and carriage returns—differences that your test really shouldn't care about. If you happen to add a tab somewhere for legibility, why should it break your test?

Usually, comparing the entire output is a bad idea: any change would break your test for bad reasons. In the test case shown in the following listing, every whitespace needs to be exactly right.

Listing 11.9 `index_test.go`: Checking the contents with strict equality

```
expect, err := os.ReadFile("testdata/index.xhtml")
require.NoError(t, err)

assert.Equal(t, string(expect), rr.Body.String()) #1
```

#1 Strict string comparison

There are better and more convoluted ways to test HTML by thinking about what exactly we're testing: we're not testing Go's `io.WriteString` function or `html` package. Frontend frameworks have thought about this thoroughly. For now, we'll stick to a golden file. We'll find quick ways to make this version work for our purpose in a satisfying way.

Run the test. You can also write an error test, to check what happens when `WriteString` blunders (and try to come up with a scenario when it would blunder).

We have some HTML output, and we've automated the test to check for changes. Don't worry, we're never going to write HTML in a `.go` file again (torture is illegal since the Geneva convention in 1987). That's where templates come in to save the day.

11.1.2 Templates: Display a variable

While the format library (`fmt`) does a great job of getting the exact format we want for a variable, it reaches its limits when we have many variables that we want to print on different lines. Suppose you wanted to display a menu in a restaurant. Each menu entry would have a name, maybe a reference number (helpful for both staff and customers), a list of ingredients, a list of allergens, maybe a suitability for vegetarians and vegans, and a price. You could use `fmt.Sprintf` for this, but the odds of your command line exceeding 80 characters are extremely high. Another option is to use a `strings.Builder`, which is a slightly better choice

but still a painful one. Imagine how difficult and complex it would be if you also want to add a translation to another language or add a calorie count somewhere.

When you tangle large portions of static text with some dynamic values, templates are a valid alternative. As we'll see in this chapter, templates provide a representation of the structured output in a very clear way—one that doesn't involve counting commas when you call `sprintf` to identify which value is printed for the fourth `%s` of the line of code.

Go comes with two libraries for templates—the `text/template` and the `html/template`. They both have the same interface; the only difference is that the HTML implementation automatically escapes HTML-sensitive characters such as `&`, `<`, or `é`. It also makes the generated text safe against code injections—preventing a text value such as `"</html>"` from ending the page being rendered by the browser or, worse, a mischievous JavaScript piece of code from being executed.

Choosing between these two libraries should always be driven by feature requirements. In this chapter, because we want to render an HTML document, we'll be using the HTML templates. Unless you need to escape some characters, which is common in XML-based languages such as HTML, using the standard `text/template` should cover all your needs. Similarly, using templates to generate any format relying on tags (e.g., JSON) or structure (e.g., YAML or CSV) should raise two questions: Is it safe to blindly trust the user's input? What is the worst that can happen if the template receives a "bad" value?

Go's templates bind a piece of code and a model. The model comes in the shape of text, usually loaded from another file, but a short template can sometimes be defined as a

variable of type `string`. Here, we'll mostly use templates from external files.

A template, after being loaded from a source and parsed (to check its syntax and grammar), will be executed on various data to produce an output. Templates in Go are based on the notion of an *action*. The template engine will apply each action in the template when it executes the template. Actions, in templates, are encapsulated by double curly braces: `{{...}}`.

Actions can be thought of as the companion to the `%v` argument in `fmt.Printf` calls. An action serves as a placeholder where we want a value to be printed. In the template, everything that isn't part of an action will be printed as such, whereas actions will be executed to whichever values they received. We therefore need to provide values to the actions in our template by using the dot `.` operator (a period character). Now that we've had enough background, let's start writing our very first template!

HTML TEMPLATE

There are a few steps we still need to go through before we can make full use of templates in Go, but we can at least start now to clean our code and extract the HTML code into a real HTML file with the `.html` file extension. This file will contain our template. Your IDE will be happier and way more useful there because it can at least help with formatting and spotting mistakes in the markup, provided it recognizes the file format from its extension. We'll resume the setup of our HTTP server right after this first example.

The first task here is to pass a variable to the template and see how it renders. Let's take the number of habits as an

example. Move the HTML string into an `index.xhtml` file, and change it using the action `{{.}}`—the default action is to print the current variable, as shown in the following listing. This is exactly the same as calling `%v` in the middle of a rather complex string.

Listing 11.10 `index.xhtml`: Outputting a variable

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Learn Go</title>
</head>
<body>
<h1>Habits</h1>
<p>You have {{.}} habits to follow.</p> #1
</body>
</html>
```

#1 Prints a variable between “have” and “habits”

SPACES

Writing `{{.}}` isn’t very clear. Instead, we can write `{{ . }}` with spaces to increase readability. These two are strictly identical; some people prefer having spaces, while some don’t. You can even use any of `{{.}}`, `{{ . }}`, `{{ . }}`, or `{{. }}` if you want. The output will be the same. Clarity, readability, and consistency are paramount here.

However, sometimes we have to add a piece of text right after the value printed from the template. Suppose we need to print a value that is a percentage. It makes sense to display it as 17.5%, with no space, rather than 17.5 %. We can use a template, and it would look like `{{.}}%` or `{{ . }}%`. In both cases, it might be difficult to see the % symbol. For this reason, templates support a feature for removing previous/next space characters. Adding a minus sign (-) on any side of the template will remove any whitespaces (newline, tab, nonbreakable space, or regular space—but, strangely enough, not an em space or an en space). When

using the `-` in the template, remember to stick it to the curly bracket and to keep it separated from the body of the action with a space: don't write `{{- .}}`, but instead `{{- . }}`. In addition, keep in mind that `-` drops all whitespaces, not just those that happened to be printed right before the `{{` characters (or right after the `}}` ones): this includes tabs and newlines that might have been written purposely. For instance, the following pairs produce the same outputs:

- `{{ . - }}` % and `{{. }}%`
- `${{ . }}` and `$ {{- . }}`
- `[ {{- . - }} ]` and `[{{. }}]`, and `[ {{- . }}` and other variations

Now that we've written our first template, we need Go to read it and fill it up. That trailing "s" in "habits to follow" in listing 11.10 will be taken care of later (it shouldn't be there when you have only one habit).

TEMPLATING IN GO

First, Go needs to read this file into a variable. We could use a file reader, but considering that the file isn't changing from one run to the next, the best tool for this is the `go:embed` tag:

```
//go:embed index.xhtml
var indexPage string
```

This syntax means that, on compilation, the contents of the `index.xhtml` file should be put into this variable. We can only use `embed` on a global variable. To understand this, the compiler requires that the `embed` package be imported, but because we're not explicitly calling any of its functions or types, we need to "silently" import it using the underscore import:

```
package server

import (
    _ "embed" // Allows us to load files
    ...
)

//go:embed index.xhtml // Load the file contents to the indexPage variable.
var indexPage string
```

If you like, you can change your unit test to use this instead of a file reader for the expected value. We can now create the template using `template.New` and give it a name. Then, we can `Parse` it from the contents of the file. The `New` function takes the name that we want to give our template, while `Parse` takes a string containing the full template. The reason templates need names will become clear by the end of this chapter, but long story short, templates can call other templates by their registered name:

```
template.New("index").Parse(indexPage)
```

Finally, we're ready to execute our template over a value. We can do this with the method `Execute`, which returns an error.

Listing 11.11 index.go: Outputting a variable

```
func (s *Server) index(w http.ResponseWriter, r *http.Request) {
    tpl, err := template.New("index").Parse(indexPage)
    if err != nil {...}

    err = tpl.Execute(w, 5) #1
    if err != nil {...}
}
```

#1 Provides a number of habits, such as 5

Habits

You have 5 habits to follow.

Figure 11.4 Result with a variable in a browser

Make sure you return an internal error and log the reason it happens. You can test the error path by providing a badly formatted template (e.g., omit a curly brace). Figure 11.4 shows the result of listing 11.11.

Update the test output, the documentation, and everything necessary to make your code future-proof. Next, we'll call the backend service because who wants a web page that never displays live data?

11.1.3 Add a gRPC client

Our HTTP server currently isn't connected to anything. To return "real" data, we need to connect it to a habits server. In a different terminal, run the `Habits` service from chapter 10. Define the port where that service listens on your machine, and write it down because you'll need it very soon. In our example, we'll use port 8084. You can use the integration test or a `grpcurl` list of commands to add some dummy habits to that service's database.

Keeping our good habit of separating concerns, let's see what packages we'll need in the HTTP server:

- `habit`—This is our core data layer, where we define the data structures that can be passed between packages.
- `client`—The anticorruption layer isolates the gRPC structures required to speak with the backend from the rest of the code and makes the calls simpler.
- `handler`—This is where the server listens to some routes and responds in HTML.

All of these are internal, inside the `internal` folder, and the `main` function will inject one into the next to get its fully functioning server. The first package, `habit`, is easy to start if we look at the same package in the gRPC service and

remove what we don't need. Listing 11.12 shows the `Habit` structure that we defined in the previous chapter; in this case, only the creation time, which isn't exposed through the API, can be discarded. What we want to expose instead is the number of times the habit was ticked this week.

Listing 11.12 `habit.go`: Defining the data object

```
// ID is the identifier of the Habit.
type ID string #1

// Name is a short string that represents the name of a Habit.
type Name string

// TickCount defines a number of ticks.
type TickCount uint

// Habit to track.
type Habit struct {
    ID          ID
    Name        Name
    WeeklyFrequency TickCount
    Ticks       TickCount #2
}
```

#1 Defines clear types

#2 Adds the ticks count

With this building block, it's possible to pass habits around between packages in our server without any of these packages' specificities leaking between them.

BACKEND CLIENT STRUCTURE

As a reminder, the client that was generated from the Protobuf definitions in the previous chapter has the following interface:

```

type HabitsClient interface {
    CreateHabit(context.Context,
        *CreateHabitRequest,
        ...grpc.CallOption) (*CreateHabitResponse, error)
    ListHabits(context.Context,
        *ListHabitsRequest,
        ...grpc.CallOption) (*ListHabitsResponse, error)
    TickHabit(context.Context,
        *TickHabitRequest,
        ...grpc.CallOption) (*TickHabitResponse, error)
    GetHabitStatus(context.Context,
        *GetHabitStatusRequest,
        ...grpc.CallOption) (*GetHabitStatusResponse, error)
}

```

We want to be able to mock it for testing purposes. This means we'll take an interface as a parameter of the `client` package's `New` function. As it happens, the interface was already generated, so why not use it? We'll be using all of these methods anyway by the end of the chapter. If we only needed a subset of methods, we would simply declare a local interface in our `client` package. Next, you'll see how to import the interface that was generated in chapter 10 into this module.

ADDING A DEPENDENCY ON A LOCAL PACKAGE

When creating a module with `go mod init <module_name>`, there are two common practices. The first popular option, mostly used for command-line applications, is to simply give the module a name, which is what we've been doing in this book: the first line of our `go.mod` file for this chapter is `learn-go-pockets/templates`. The other choice is to use the full domain name of the module, for instance, `k8s.io/kubernetes` or `github.com/grafana/grafana`. This option is mostly used for libraries, or modules that are intended to be shared, but it can very much be used for applications too, as these examples show.

Now, we're faced with a problem—because our module for chapter 10 wasn't published and exposed, how can we

access its files from the module of this chapter? How do we access the generated gRPC server definition, the one that lives in the `learn-go-pockets/habits` module?

If we try to add a dependency to it using the standard command, we get an error:

```
> go get learn-go-pockets/habits
go: learn-go-pockets/habits@v0.0.0-00010101000000-000000000000:
malformed module path "learn-go-pockets/habits": missing dot in
first path element
```

As you can see, Go expects a `.` in the name of the module we're trying to import—the separator between the host name and the top-level domain name. We could try and import a local file by using the relative path to the `.go` file, such as `import "../habits/api"`. However, this also won't end happily, as trying to solve the imports afterward will result in an error:

```
> go mod tidy
../habits/api/: "../habits/api" is relative, but relative import
paths are not supported in module mode
```

So, to solve this conundrum, we can use `go mod edit -replace` to override the version or the path of a module. This has two main uses: when we want to specify which version/tag/branch of the library is to be used, or when we want to refer to a local implementation. The syntax is as follows:

```
> go mod edit -replace=old_module=new_module
```

For example, if we wanted to use a local clone of `github.com/stretchr/testify` that we stored at `../stretchr/testify` (could also be an absolute path), we would use the following:

```
> go mod edit -replace=github.com/stretchr/testify=../stretchr/testify
```

To specify the version/branch/tag to use, simply add @ and the reference:

```
> go mod edit -replace=github.com/moby/moby=github.com/moby/moby@dev
```

In our case, we want to use a version of the module that is stored locally in a different path. We need to provide the path to the module (not the package), which, in the case of our implementation, is the following:

```
> go mod edit -replace=learn-go-pockets/habits=../10-habits/7_track
```

If you haven't implemented chapter 10, you can clone it from our repository and then run the command with the appropriate path.

Now, if we take a look at the `go.mod` file, it should have a line (usually at the end) showing the effect of the `replace` command: `replace learn-go-pockets/habits => ../10-habits/7_track`. Great! We can now import the `api` package of the `learn-go-pockets/habits` module—Go should now be able to import the local package. However, remember that this adds a dependency to a local file, so sharing this with colleagues will very likely cause trouble. For this reason, it's almost always a better option to publish modules prefixed by a domain name (e.g., `github.com`), from where they can be retrieved by the Go toolchain.

IMPLEMENTING THE GRPC CLIENT

Now that we can access the definition of the backend service, let's create a new client structure in the `internal/client` package with one field: the gRPC client that we want to hide from the rest of the service.

Listing 11.13 `client.go`: A wrapping client

```
// HabitsClient is a wrapper around the gRPC client.
type HabitsClient struct {
    cli api.HabitsClient #1
}

func New(cli api.HabitsClient) *HabitsClient {
    return &HabitsClient{cli: cli} #2
}
```

#1 gRPC client

#2 Builds by taking the client as parameter

We want our HTML document to list all registered habits and their statuses, as well as offer the possibility to tick a habit with a click. This isn't a single feature, and we need to start somewhere. It makes sense to start by retrieving all the habits registered, as we'll then be able to use their ID to perform other actions.

We want this client to retrieve the registered habits, so we'll add a `ListHabits` method on it, as shown in listing 11.14. Using the hexagonal approach, which recommends clustering code into independent packages, the types generated by the `proto` toolchain shouldn't be exposed outside this `client` package, as its API must only rely on the local `habit` package for domain type definitions.

Listing 11.14 list.go: Client can list the habits

```
// ListHabits lists the habits available.
func (hc *HabitsClient) ListHabits(ctx context.Context,
    t time.Time) ([]habit.Habit, error) { #1
    resp, err := hc.cli.ListHabits(ctx,
        &api.ListHabitsRequest{}) #2
    if err != nil {...}

    list := make([]habit.Habit, len(resp.Habits))
    for i, h := range resp.Habits {
        list[i] = habit.Habit{ #3
            ID: //...
        }
    }
    return list, nil
}
```

#1 Provides a timestamp for the tick count

#2 Calls the backend

#3 Converts from gRPC to the domain type

We also need to know for each habit how many times it has been ticked during the week of the timestamp. To express times, use the `timestamppb` package for Protobuf-compatible timestamps. Add the dependency with `go get`:

```
> go get google.golang.org/protobuf/types/known/timestamppb
```

The number of ticks for that week is available, but only on a per-habit basis: it's the `GetHabitStatus` endpoint. That means we need another call inside the loop.

Listing 11.15 list.go: Getting the number of ticks

```
import "google.golang.org/protobuf/types/known/timestamppb"

...

for i, h := range resp.Habits {
    status, err := hc.cli.GetHabitStatus(ctx, #1
        &api.GetHabitStatusRequest{
            HabitId: h.Id,
            Timestamp: timestamppb.New(t),
        })
    if err != nil {...}

    list[i] = habit.Habit{
        ID: //...
        Ticks: habit.TickCount(status.TicksCount),
    }
}
```

#1 Gets each habit's status

Now would be a good moment to wonder whether a ListHabits endpoint that already returns the ticks as part of the gRPC API of the backend is appropriate or if a bulk GetHabitStatuses that gets the statuses of a whole list of habits in one call would be better. The rationale behind our design is that optimizing too soon often leads to optimizing in the wrong places. Donald Knuth, author of *The Art of Computer Programming*, said the following:

We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%.

Users will rarely have loads of habits that would generate hundreds of similar calls; if they do, maybe paginating is a better idea than adding complexity to the backend service. If you run this code in production, and your monitoring tools show that a lot of your resources are spent fetching statuses and hampering the performance of your application, then one of the two options is worth exploring.

Here, we could add an extra layer of logic to cover timeouts when contacting the backend service. Because this isn't the purpose of this chapter—and because it's already been covered in chapter 10, we'll blindly trust that our connection to the backend is reliable. Just don't do this in a production environment!

There are only three paths for testing: success, error while fetching the list, and error while fetching the statuses. Let's write these test functions—one with a mock that returns some data, and two where the mock returns an error. Again, we'll be using `minimock` to generate the mocks, as we did in chapter 5.

Note the use of `minimock's Expect` when we only expect one call, and the slightly more verbose `Set` when we need multiple responses depending on the input.

Listing 11.16 list_test.go: Testing the happy path

```
package client_test

//go:generate minimock -i
↳learngo-pockets/habits/api.HabitsClient -s "_mock.go" #1

func TestListHabits(t *testing.T) {
    mockClient := mocks.NewHabitsClientMock(t) #2

    mockClient.ListHabitsMock.Expect( #3
        minimock.AnyContext, #4
        &api.ListHabitsRequest{}).
    Return(&api.ListHabitsResponse{
        Habits: // ...,
    }, nil) #5
    mockClient.GetHabitStatusMock.Set(getHabitMock)

    habitsClient := client.New(mockClient) #6
    habits, err := habitsClient.ListHabits(context.Background())

    require.Nil(t, err) #7
    expectedHabits := //...
    assert.Equal(t, expectedHabits, habits)
}

func getHabitMock(_ context.Context,
    in *api.GetHabitStatusRequest,
    ...grpc.CallOption) (*api.GetHabitStatusResponse, error) {
    if in.HabitId == "ID1" {
        return &api.GetHabitStatusResponse{TicksCount: 3}, nil
    }
    // ...
    return nil, fmt.Errorf("unexpected ID")
}
```

#1 Generates a mock from the external interface

#2 Creates a mock instance

#3 Mocks the call that returns some habits

#4 Any context is valid in this situation.

#5 Mocks the call for get status

#6 Instantiates the client

#7 Checks that everything went well

Because this tests the behavior of the client's exposed function, it makes sense to not write it as an internal test. For this reason, the package for this test file is `client_test`. This test file needs to import the package containing the implementation, `learngo-pockets/templates/internal/client`. In the following listing, let's have a go at testing the error path, where we don't need to mock the `GetStatus` call at all.

Listing 11.17 list_test.go: Testing when listing returns an error

```
func TestListHabits_error(t *testing.T) {
    sentinelErr := fmt.Errorf("list habits error") #1

    mockClient := mocks.NewHabitsClientMock(t)
    mockClient.ListHabitsMock.Expect(minimock.AnyContext,
        &api.ListHabitsRequest{}).Return(nil, sentinelErr)

    habitsClient := client.New(mockClient) #2
    habits, err := habitsClient.ListHabits(context.Background())

    require.ErrorIs(t, err, sentinelErr) #3
    assert.Nil(t, habits)
}
```

#1 Defines the expected error

#2 Instantiates the client with the error-sending mock

#3 Asserts that we get the error

Finally, the third test covers the case where one of the calls to `GetStatus` fails. The mock for `ListHabits` returns some good data, but the mock for the second status returns an error. While some mock tools allow calling an equivalent of `Expect(EndpointName, ExpectedParameters)` several times per endpoint, `minimock` doesn't. Instead, we need to override the behavior of the mock. For this, we use the `Set` method and provide it with our fake implementation that we can adapt to the calls we'll make.

Listing 11.18 list_test.go: Testing when get status returns an error

```
func TestListHabits_statuserror(t *testing.T) {
    now := time.Now()
    mockClient := mocks.NewHabitsClientMock(t)
    habitsClient := client.New(mockClient)

    mockResponse := &api.ListHabitsResponse{
        Habits: []*api.Habit{
            {Id: "ID1", Name: ...},
            {Id: "ID2", Name: ...},
        },
    }

    sentinelErr := status.Error(codes.Internal, "not after 10PM")
    mockClient.ListHabitsMock.
        Expect(minimock.AnyContext, &api.ListHabitsRequest{}).
        Return(mockResponse, nil) #1

    mockClient.GetHabitStatusMock.Set(
        func(ctx context.Context,
            in *api.GetHabitStatusRequest,
            opts ...grpc.CallOption) (
                *api.GetHabitStatusResponse, error) {
            switch {
            case in.HabitId == "ID1":
                return &api.GetHabitStatusResponse{}, nil #2
            case in.HabitId == "ID2":
                return nil, sentinelErr #3
            default:
                return nil, fmt.Errorf("unexpected call")
            }
        })

    habits, err := habitsClient.ListHabits(context.Background(), now)

    require.ErrorIs(t, err, sentinelErr) #4
    assert.Nil(t, habits)
}
```

#1 Sets the output of the call to ListHabits

#2 The first GetStatus succeeds.

#3 The second GetStatus fails.

#4 The overall response is an error.

We've now tested client's ListHabits logic. To use it, we need the main function to build the dependencies required by our server.

DEPENDENCY INJECTION

On one side, we're writing an HTTP server. On the other side, we're using a gRPC client to retrieve habits. The layer

in charge of instantiating these two and plugging them together is the `main` function, which will inject the dependencies in the layers below it as needed. The `server` package shouldn't use the API generated by the `proto` tools. Instead, it should use its own small interface of what a `habit` client is—that's generally the approach in Go.

The `main` function will need to create a gRPC client and feed it to the HTTP server as shown in listing 11.19. For this, we'll instantiate the gRPC client as we did in the integration tests section of chapter 10. Usually, the remote dependency's address won't be `localhost:8084` (unless you're very lucky). It should be possible to inject the other service's address via a configuration file, a Kubernetes configuration map, a Docker network setting, or whatever the platform uses.

Listing 11.19 `main.go`: Building the client

```
func main() {
    cli, err := newClient("localhost:8084") #1
    if err != nil {
        log.Fatalf(...) #2
    }

    srv := handlers.New(cli)
    //...
}

// newClient creates a client to the habits service.
func newClient(serverAddress string) (*client.HabitsClient, error) {
    creds := grpc.WithTransportCredentials(insecure.NewCredentials())
    conn, err := grpc.Dial(serverAddress, creds) #3
    if err != nil {
        return nil, err
    }

    grpCli := api.NewHabitsClient(conn)

    return client.New(grpCli), nil
}
```

#1 Includes the backend service's port

#2 No point in going any further

#3 A lot of dial options are available; check go doc `grpc.DialOption` for a list.

The HTTP server needs to know what to do with this new parameter. Let's update the code of the `New` function in the server package, and define the small easy-to-mock interface we'll use from the server's point of view, as shown in listing 11.20. Thanks to our interface, it should now be clear that the HTTP server is totally unaware that retrieving habits requires a gRPC client under the hood.

Listing 11.20 server.go: Using the client dependency

```
// HabitsClient is the dependency towards the Habits client.
//go:generate minimock -s "_mock.go" -o "mocks" #1
type HabitsClient interface {
    ListHabits(ctx context.Context) ([]habit.Habit, error)
}

// Server serves all the HTML routes on this service.
type Server struct {
    client HabitsClient #2
    lgr     Logger
}

// New builds a new server.
func New(cli HabitsClient, lgr Logger) *Server { #3
    return &Server{
        client: cli,
        lgr:    lgr,
    }
}

// Router returns an HTTP handler that listens to all the proper paths.
func (s *Server) Router() http.Handler {...}
```

#1 Mocks the dependency for tests
#2 Adds a field in the server struct
#3 Requires the dependency

Finally, let's return to the `index` method. It now has access to the backend client via its receiver `s` and can call it to get the habits. Hopefully, everything will go smoothly when the HTTP server calls the gRPC server, but in case of an error, we should display some pretty 5xx (internal server errors) message to the client, as shown in the following listing. A Retry button would be cool, but let's keep it simple and assume that people know how to refresh a page.

Listing 11.21 index.go: In case of an error

```
// index serves the root page of the app.
func (s *Server) index(w http.ResponseWriter, r *http.Request) {
    habits, err := s.client.ListHabits(r.Context()) #1
    if err != nil {
        s.lgr.Logf("error! %s", err.Error()) #2
        http.Error(w, "Error while fetching data",
            http.StatusInternalServerError) #3
        return
    }
    ...
}
```

#1 Uses the request context

#2 Logs the error

#3 Returns an obfuscated error, as we haven't called Write on w yet

Habits

You have 2 habit(s) to follow.

Figure 11.5 Browser result with a variable

Of course, we could be more specific and check inside the client package of the RPC return code that we get, turn it into a named error, and there check for this error. At the moment, there's no situation where a different code returned by the backend service would mean anything but Internal Problems, so this piece of code will do the work. If, for example, you decide to add some authentication on the habits server, then returning a 401 instead when relevant can make sense.

Replace the hardcoded value 5 in the call to `Execute` on our template with `len(habits)`. Figure 11.5 shows the result.

We have an extra "s" in the generated page at "habits" and no details on our habits themselves. It's time to provide some visibility and play with lists. Let's crack open the power of templates!

11.2 Basic template operations

You've seen how to insert one variable inside the HTML template. In this section, we'll go through operations such as accessing fields and expressing conditions with templates. We'll see how we can display the details of a slice of habits. Templates in Go are engines that update a string's contents using values. The following listing shows an example of using a template.

Listing 11.22 `index.xhtml`: Unordered list in the template

```
package main

import (
    "fmt"
    "text/template" #1
    "os"
)

func main() {
    tmpl := `{{ range . }}{{.}}, {{end}}` #2
    t, err := template.New("my_template").Parse(tmpl) #3
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        return
    }

    values := []string{"a", "b", "c"}
    err = t.Execute(os.Stdout, values) #4
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        return
    }
}
```

#1 Imports `text/template` or `html/template` depending on the overall goal

#2 Defines the template

#3 Parses the definition into a usable template

#4 Applies the template on a string and writes it somewhere

This section will help you understand templates and explain how to start working with them.

11.2.1 Iterate through a slice

Templates can do a bit more than just print a variable, as they also have control flow statements. To iterate through a list of values, templates support the `range` action. Remember that in our current setup, the `habits slice` is the variable that we pass to our template execution, which means it's represented by the dot `.` as in the following:

```
{{ range $index, $value := . }} <some html> {{ end }}
```

Don't forget that the `{{ end }}` (or `{{end}}`) is the equivalent of Go's closing curly bracket in a `for` loop. The input value can be a `slice`, `map`, `channel`, `array`, or anything that Go knows how to `range` over. Using `$index` and `$value` creates variables that are usable in the scope of the `{{range}}...{{end}}` block. The action to print the index, for instance, is `{{ $index }}`.

Alternatively, if you're not using the `index` variable in the template, you can use `{{ range . }}` ... `{{ end }}`, and the value of the dot variable inside the action will iterate through the values of the `slice`. This might be counterintuitive, as when we only iterate over a `slice` or `map` in Go with the `for x := range values` syntax, `x` is either the index in the `slice` or a key of the `map`. With `{{ range . }}`, we iterate through elements of the `slice` or values of the `map`. Let's see it in action first with just a `slice` of your favorite numbers.

If we want to show a list of `index : number` as a list of items in a web page, our source HTML won't differ much from the syntax described previously. Let's add the following snippet in the `<body>` section of our `index.xhtml` document.

Listing 11.23 index.xhtml: Unordered list in the template

```
<ul>
  {{ range $index, $value := . }} #1
  <li>{{ $index }} : {{ $value }}</li>
  {{ end }}
</ul>
```

#1 Loops over some data

We're using the more verbose version because we want to show the `index` variable. Then, we just pass the `slice` to the template.

Listing 11.24 index.go: Outputting a slice

```
values := []int{47, 52, 88, 18}

err = tpl.Execute(w, values) #1
if err != nil {...}
```

#1 Passes the slice as parameter to the template

Slice Viewer

Here is a slice:

- 0 : 47
- 1 : 52
- 2 : 88
- 3 : 18

Figure 11.6 Browser result with a slice of ints

The result will look a bit stark, but in a very few lines, we have a working demo. Figure 11.6 shows the result in a browser.

Now that we've checked the `{{range}}` `{{end}}` with a slice of integers, let's return to our habits, and try to print the list of one habit's registered objectives.

11.2.2 Accessing fields

We want to display the name and weekly target frequency for each habit. In an unordered list (`ul`), each list item (`li`) could display the name in bold (`b`) and the number of times a week, as shown in figure 11.7. Some say that bold shouldn't be a tag but should be a styling property, and we should replace this with a `span`, but let's keep it old school for the sake of clarity. After all, this chapter is about Go's templates, not HTML syntax.

Habits

Here are your registered habits:

- *Knit*—5 times a week

Figure 11.7 Browser result with a list of habits

The static HTML code for this example looks straightforward:

```
<ul>
  <li>
    <b>Knit</b> - 5 times a week
  </li>
</ul>
```

To get this result, our template is already ranging over the habits. We're using the shorthand version where `.` represents the slice of habits in `{{range .}}`, and, inside this range, the dot `.` is the habit currently being iterated over (see listing 11.25). To access the fields of this specific habit, just append the name of the field after the symbol. It absolutely needs to be exposed! This means that the template and the codebase are heavily intertwined. Changing the name of a field or making it unexposed will result in it no longer being printed.

Listing 11.25 index.xhtml: Accessing fields

```
<ul>
  {{- range .}} #1
  <li>
    <b>{{ .Name }}</b> - {{ .WeeklyFrequency }} #2
    time(s) a week
  </li>
  {{- end}}
</ul>
```

#1 This dot is the slice.

#2 These dots are the current element. We access the habit's Name and WeeklyFrequency.

Give the `slice` of habits to the template for execution, and you can run it:

```
err = tpl.Execute(w, habits)
```

Do you see all of them? Does your test break? Are you struggling with the whitespaces in the test? Remember that using `{{- or -}}` can help reduce the whitespaces, new lines, and tabs.

You can already play around and show different fields of the habits. While it's very tempting to try and get the output of a method (after all, aren't they exposed from a type, just like fields?), a fair warning is recommended here: calling methods in a template is possible with two main requirements:

- The method must be niladic (receive no arguments)
- The method must return one value (or two, if the second one is an error). In this case, the (first) returned value will be printed.

If the method ticks the box, it's possible to call it in the template with no parentheses: `{{ .Method }}`. It's even possible, as suggested by the official documentation, to chain the calls, something we don't recommend doing at all, for clarity: `{{ .Field1.Method1.Field2.Method2 }}`. Please don't

do this. Instead, use a function or a dedicated template, as we'll see later.

Before we move on to more functionalities for our users, let's see how we can fix the lowercase "s" in "habit(s)" that currently requires parentheses.

11.2.3 Conditional formatting

Sometimes, there are panels that you want to show on certain conditions. The good news is you can use an `if` pattern inside the templates. There are a handful of keywords for this, just like in Go:

```
{{ if <condition> }}  
{{ else if <condition> }}  
{{ else }}  
{{ end }}
```

The `end` keyword is the same that we use to close the scope of a `range` statement.

The conditions are a bit different: they use a pattern where the comparing operation comes first, and the operands come next. Comparison operators are strict: `lt` stands for "(strictly) less than," and `gte` stands for "greater than or equal to." You can find more Go templates conditions in table 11.1.

Table 11.1 Conditions

Go code	Go template
<code>if h (Boolean)</code>	<code>{{ if . }}</code>
<code>if h != 0 (any numeric type, including rune)</code>	<code>{{ if . }}</code>
<code>if h != "" (string)</code>	<code>{{ if . }}</code>
<code>if len(h) != 0 (slice, map, array, string, <i>not</i> channel)</code>	<code>{{ if . }}</code>
<code>if h != nil (interface, uninitialized slice, map, channel, pointer)</code>	<code>{{ if . }}</code>
<code>if h < 3 (numerical value)</code>	<code>{{ if lt . 3 }}</code>
<code>if h == "pockets"</code>	<code>{{ if eq . "pockets" }}</code>
<code>if h >= -8</code>	<code>{{ if gte . -8 }}</code>
<code>if h != 4</code>	<code>{{ if ne . 4 }}</code>
<code>if s.X < 40 && !s.End</code>	<code>{{ if and (lt s.X 40) (not .End) }}</code>

With the `{{if}}` action, using a singular form for the number of times a week becomes possible. Note the use of `-` to remove extra spaces:

```
{{ .WeeklyFrequency }} time{{ if gt .WeeklyFrequency 1 -}} s {{- end }} a week
```

You can even use pretty words like once or twice:

```
{{ if eq .WeeklyFrequency 1 }} once {{ else if eq .WeeklyFrequency 2 }}  
twice {{ else }} {{ .WeeklyFrequency }} times {{ end }} a week
```

The result isn't necessarily easier to read for the end user (ask a UX designer), but it works: you can see it in figure 11.8. This kind of complex logic can be extracted into a function, as we'll see in section 11.5.2.

We can now display a web page containing the real data living in our database. However, all we can do so far is access it for reading—as a user, it would be quite frustrating to not be able to register some of these activities! Before we move on to our next step, don't forget to test. If the mocked call to the `Habits` service already provides some data, you only need to update the test output. Make sure you also have error cases covered.

Habits

Here are your registered habits:

- *Knit*—2 out of 5 [Tick!](#)
- *Call mom*—1 out of 1 [Tick!](#)
- *Code*—0 out of 3 [Tick!](#)

Figure 11.8 The list of habits with their associated frequencies per week

Commit your code. It's time to tell the system when some of this to-do list gets done.

11.3 Send a tick to the server

Having a to-do list is a good way to get things out of your mind and onto paper, but ticking the boxes is the only way

to make sure the day feels productive. Cognitive research suggests that keeping a “Done” list is healthy for people who highly value productivity, as it provides a sense of accomplishment. It’s better for your brain than keeping a list in view of what you still haven’t tackled after all this time.

11.3.1 What page should we display?

In this chapter, we decided to have all the logic happen server-side, as opposed to inside the browser. This means that to send information or a command to the server (e.g., “tick habit with ID 987 now”), the only action we can ask the user’s browser to take is to go to another page. An alternative is to use JavaScript to execute requests to the HTTP server.

Because the “HT” in HTML stands for Hypertext, we can use this ancient magic where a piece of text can help navigate to another page. The goal here is to implement an HTTP endpoint that ticks a habit, given its ID. The target URL is `/tick/{habitID}`. Let’s first add such a link in each of the habits in our list.

Listing 11.26 `index.xhtml`: Adding a link to tick each habit

```
<li>
...
  <a href="/tick/{ { .ID } }">Tick!</a> #1
</li>
```

#1 The HTTP link

If you click it, the browser will redirect to this link and try to GET the page. As this is an action, we should probably use a POST verb. We’ll do this in the next section. In the meanwhile, we’ll use a GET here, as shown in listing 11.27. Let’s add this route to our service, to avoid the classic 404. As we’ve seen in previous chapters, the standard `html` library understands a path parameter as a string inside curly brackets.

Listing 11.27 server.go: Registering the tick endpoint route

```
func (s *Server) Router() http.Handler {  
    // ...  
    r.HandleFunc(http.MethodGet+" "+"/tick/{habitID}", #1  
        s.tick)  
    return r  
}
```

#1 Registers the route with the URL parameter habitID

Now we need to implement this new `tick` method on the server. The signature is imposed: the `http` package requires it to implement an `http.HandlerFunc`. First, the endpoint needs to parse the identifier from the request's URL using the `http.Request`'s `PathValue` method. Second, we can call the client—let's keep this bit for last. Write this endpoint in a new file, `tick.go`.

Listing 11.28 tick.go: Ticking a habit

```
// tick adds a tick to the given habit.  
func (s *Server) tick(w http.ResponseWriter, r *http.Request) {  
    const (  
        habitIDPathValue = "habitID"  
    )  
  
    id := r.PathValue(habitIDPathValue) #1  
    if id == "" {  
        http.Error(w, "missing the id of the habit", http.StatusNotFound)  
        return  
    }  
  
    // TODO call the gRPC client  
}
```

#1 Uses the name of the url parameter

Next, we need to provide some feedback to the user. We could write something like “Well done, click here to return to the main page,” but we don't want to worsen any possible carpal tunnel syndrome by asking for more clicks. An alternative is to display the index discreetly, after the backend's `Tick` endpoint has been called:

```
s.index(w, r)
```

Doing so would work, so you can try it. The user navigates to `/tick/ID` and sees the index again, but with the new data. But if the user refreshes the page, there will be another tick marked, which is counterintuitive.

Fortunately, we're not the first developers to need a redirect after sending some command to a server. The HTTP status 303, delicately described as *See Other*, seems to have been made for us:

```
http.Redirect(w, r, "/", http.StatusSeeOther)
```

This tells the browser to go back to the previous page, which, in our case, was the index. Here, you can see the point of using a constant for the main page's path: redirecting to the constant called `indexPath` will make your code clearer than the mysterious `"/"` in the preceding example.

We can already test this part: reading the `habitID`. You'll need, as usual, a `ResponseRecorder`. You also need to provide our request with the URL parameter by setting the value with the `SetPathValue` method. Let's write some tests in `handlers/tick_test.go`.

Listing 11.29 tick_test.go: Testing with URL parameters

```
func TestServer_Tick(t *testing.T) {
    rr := httptest.NewRecorder()

    req := httptest.NewRequest(#1
        http.MethodGet, "/tick/", nil)
    req.SetPathValue("habitID", "1234") #2

    s := New(nil, t)

    s.tick(rr, req)

    assert.Equal(t,
        http.StatusSeeOther, rr.Result().StatusCode) #3
    assert.Contains(t, rr.Body.String(), `<a href="/>`) #3
}
```

#1 Creates the request as usual

#2 Provides the URL parameter

#3 Validates the status and redirection

In this test, we're not testing the full flow, but providing a request with some already prepared data. This might look like a cheat, but this is a unit test, and we don't want to test the `PathValue` logic and what happens before the `tick` method is called. If we really wanted to test the entire logic, then an integration test is necessary, where we would run the service and call it like any client would do. We chose to stay within the scope of a unit test. This endpoint is nice but it doesn't do its job yet, so we need to call the gRPC client.

11.3.2 Send tick to the backend

Finally, we need to connect the client to the backend `Habits` server. Using the same pattern as `ListHabits` does, write a `TickHabit` method on the `HabitsClient`. The method takes a `context.Context` and a habit ID, and it can return an error. It hides the call to the gRPC client. Because the backend's response for `TickHabit` is empty, we can keep only the returned error, if any.

The backend's request, however, requires a timestamp for when the habit was ticked. How are we telling the client

which week we want? Let's first hardcode the current date in the next listing, and come back later (in section 11.5.1) to add a parameter.

Listing 11.30 tick_test.go: Testing with URL parameters

```
// TickHabit adds a tick now.
func (hc *HabitsClient) TickHabit(ctx context.Context, id habit.ID) error {
    req := api.TickHabitRequest{
        HabitId:    string(id),
        Timestamp:   timestamppb.Now(), #1
    }

    _, err := hc.cli.TickHabit(ctx, &req) #2
    return err
}
```

#1 A Protobuf-compatible version of the current timestamp

#2 Calls the Habits service

As you can see, if the API changes and the backend server starts returning something useful in the response, this is the layer that will decide how to handle it and whether to expose it. Before we can use this new method, we need to update our `HabitsClient` interface server-side, as shown in listing 11.31. Our server's dependency to the client layer only exposes a single method—`ListHabits`. Let's add the new `TickHabit` there and regenerate the mocks. Finally, call this client inside the `tick` method in the server layer, write a test with a pretty mock, run it, and enjoy. We can now display the habits we registered, and we can click our page to record the progress.

Listing 11.31 server.go: Adding the `TickHabit` method in the interface

```
//go:generate minimock -s "_mock.go" -o "mocks"
type HabitsClient interface {
    ListHabits(ctx context.Context, t time.Time) ([]habit.Habit, error)
    TickHabit(ctx context.Context, id habit.ID) error
}
```

Add the client's mock to your unit test, and check that it does receive the proper `Habit ID`. Running the server and clicking on the `Tick!` link should now update the

counter. While this is great, we can improve on the user experience by adding visual feedback, improving on the data already displayed.

11.3.3 Adding colors

HTML has a best friend that grants it pretty colors: Cascading Style Sheets, commonly referred to as CSS. They are called *cascading* because properties of the containers are inherited from their parent container. To cut the frontend theory short, we can add a class to any HTML tag and apply some styling to it. This won't add much to the code of the list items (or any other container), but it will change the way they are rendered in the browser window.

Tip We recommend *CSS in Depth* (Manning, 2024, www.manning.com/books/css-in-depth-second-edition) by Keith J. Grant if you want to learn more about CSS.

Let's create a CSS class called `done` for habits whose number of ticks have reached the targeted weekly frequency, and let's add a class attribute to the list items. To compare if the goal has been met, we'll use an `{{ if ... }}` template, comparing the tick count to the objective.

Note that if you code more than the target of three times a week, it still counts as completed; basically, any number of times greater or equal to the target works:

```
<li class="{{ if gte .Ticks .WeeklyFrequency }}done{{end}}">
```

Actually, this logic should be in Go, as it will be much easier to test: a `Habit` variable should be able to tell whether it's completed or not. Write that method (remember, it needs to be exposed for the template to access it), write a test for it, and you can add an `IsDone` method on our `habit`, as shown in

listing 11.32, and call this new method in the template instead of using the preceding line of code:

```
<li class="{{ if .IsDone }}done{{end}}">
```

Listing 11.32 habit.go: Exposing whether a habit was completed

```
// IsDone returns whether a habit has been fully completed.
func (h *Habit) IsDone() bool {
    return h.Ticks >= h.WeeklyFrequency
}
```

Add the following styling code inside the `<head>` tag to show the difference (we're keeping the styling very minimal, not indicating how CSS should be written):

```
<head>
...
<style>
  .done {
    color: white;
    background-color: darkgreen;
  }
</style>
...
</head>
```

Feel free to change the colors to make them colorblind friendly and to match your own taste. Restart the server, and check the result: Does ticking a habit enough times to reach the objective change its appearance? Update the test, and make sure it runs smoothly and covers all the relevant cases.

11.4 Use a form to create a habit

In software development, we always try to ship little increments to the users, so that we can align the product with their needs. More concretely, adding small changes to a product also allows it to be deployed early and generate revenue as soon as possible. When making an idea happen, be it a full product or a single feature, we start by looking

for the minimal viable product, eliminating as many features as possible, and then iterating on the users' feedback. We've already pushed the feature of ticking a habit at a specific date to a later time, but there's one feature without which we can't ship our product: creating a habit.

11.4.1 HTML form

To create a habit, we need to retrieve some values from the user, such as the habit's name or its frequency. For this, we'll ask the user to fill in a form, and provide a way of "sending" the habit to the server to have it created. HTML has the `form` tag for sending data to a server. Among the different attributes `form` can take, we'll focus on `action`, which determines the path where the data should be sent, and `method`, which determines the verb that will be used:

```
<form action="/create" method="post">
```

We want to create a habit, so `POST /create` seems pretty straightforward. If we were also dealing with other kinds of entities, it would be polite to prefix the path with the word `habits`, but let's keep it simple.

A `form` element can itself contain several other elements, each of them with a specific purpose. For our needs, we'll only use three of them: the `input`, the `label` to parse input values, and a `button`, in charge of making the call to the server.

From a server's perspective, we only need each field to be attached to a name that will help us discriminate between them. That's where the `input` tag is used—it's an empty field in which the user can write data. From a user's perspective, though, we need to display some text to tell the user what data we expect in our different inputs, and that's the role of

the label. A label's for attribute needs to correspond to the input's id.

Some programs, such as readers for people with visual impairments, use labels extensively. You can read more about the Web Accessibility Initiative (WAI) on the W3C website. Finally, a button will send the form's data to the server when clicked. The following listing shows the contents of our form.

Listing 11.33 index.xhtml: Form to create a habit

```
<form action="/create" method="post">
  <label for="habitName">Objective:</label> #1
  <input type="text" id="habitName" name="habitName">

  <label for="habitFrequency">Weekly frequency:</label> #2
  <input type="number" id="habitFrequency" name="habitFrequency">

  <button>Create</button>
</form>
```

#1 Matches the for attribute of the habitName label to the habitName input's id value

#2 Matches the for attribute of the habitFrequency label to the habitFrequency input's id value

Check the result in your browser and add some pretty styling if you like. Figure 11.9 shows the result, and as you can see, we didn't style it.

Habits Viewer

Objective: Weekly frequency:

[Prev](#) This period covers 20 October 2024 through 26 October 2024. [Next](#)

Here are your registered habits (you have three):

- *Knit*—2 out of 5 [Tick!](#)
- *Call mom*—1 out of 1 [Tick!](#)
- *Code*—0 out of 3 [Tick!](#)

Figure 11.9 Browser result with a form

11.4.2 Read a form's values

We defined the `/create` route where the data is sent when the button is clicked, so now let's listen to it on the server's end. To implement the whole chain of calls, we'll need to add a call to the backend's `CreateHabit` endpoint. Let's start with the `internal/client/create.go`. The implementation should be straightforward: we receive a habit's definition, we convert it to its Protobuf representation, and we call the backend's endpoint. Test it with a mocked gRPC client using the same strategy as we used with `TickHabit`.

Then, we need to add a handler similar to `tick` on the server. The data is accessible via the `FormValue` method on `http.Request`, which only returns strings. Use the `strconv` package to parse the target frequency as an integer.

Listing 11.34 create.go: Parsing form values

```
// create takes a JSON request and creates a Habit from it,  
// then redirects to index.  
func (s *Server) create(w http.ResponseWriter, r *http.Request) {  
    habitName := r.FormValue("habitName") #1  
    weeklyFreq, err := strconv.Atoi(  
        r.FormValue("habitFrequency")) #2  
    if err != nil {...}  
  
    // call client  
  
    // redirect to index  
    http.Redirect(w, r, "/", http.StatusSeeOther) #3  
}
```

#1 Reads a string

#2 Reads an int

#3 Creating a new habit should refresh the page.

Finally, we need to update the router to register the new POST endpoint:

```
r.Post("/create", s.create)
```

Who should validate the fields of the new habit? Should it be the role of the HTTP server—after all, an early check would save time on useless calls. Should it only be the responsibility of the backend—doesn't the backend know what a "valid name for a habit" should be? We decided to not have this logic in the frontend. However, this shows we need a safety net here to protect the backend against nasty attacks. For instance, we could make sure the weekly frequency is meaningful—maybe something between 1 and 100. Similarly, the habit's name should be a string of between 1 and 200 runes in length.

For now, if the user inputs invalid data, we don't really have a way of sending the user a pop-up explaining what went wrong. We'll continue logging to `os.Stdout` from the server. This chapter is about templates, so let's explore what extra possibilities they offer.

11.5 More template niceties

We've seen the most common tags that can be found in templates and how to use them. There are also a handful of other ways of using templates in Golang.

11.5.1 Passing more than one object to the template

The list of habits isn't the only data we want to pass to the template upon execution. The product, so to speak, is designed to be based on weeks, yet we don't display the date and can't navigate through the previous or future weeks.

Let's see how we can display the week's start and end dates. The constraint here is that we have one template document (our HTML page), and we'd like to pass it several values, instead of solely the current list of habits. Let's first decide what we want to display, and then look at a way to achieve it.

FORMATTED WEEK TYPE

Passing a timestamp directly to the template and having it printed in the web page wouldn't be very user friendly. Instead, we want to display something like "Week from June 30 to July 6." For a simpler code, we'll make the week start on a Sunday. The reason for this is that the `time.Weekday`'s zero value is `time.Sunday`—using a Monday or a Saturday (the other usual first days of the week), a Friday (in the Maldives), or any other day would require a little arithmetic.

We can create a type built from a timestamp and expose two methods: `Start` and `End` of the week containing this

timestamp, as shown in the following listing. This type can live in the `internal/habit` package with all our business types (for the moment just the one: `Habit`).

Listing 11.35 `week.go`: API

```
// FormattedWeek defines the start and end of a week, and formats them.
// Start to end will be from Sunday 00:00 to Saturday 23:59,
// rounded to the minute.
type FormattedWeek struct {} #1

// NewFormattedWeek builds an immutable week from one moment inside it.
func NewFormattedWeek(include time.Time) FormattedWeek #2

// Start returns the formatted first day of the week.
func (w FormattedWeek) Start() string #3

// End returns the formatted last day of the week.
func (w FormattedWeek) End() string#3
```

#1 Exposed type

#2 Builds from a provided timestamp

#3 Methods that return only a string can be called in the template.

We have two options here: keep the provided timestamp and compute the start and end when they are needed, or compute the start and end in the `NewFormattedWeek` function and return them when they are needed. The latter forces us to create an immutable type, which is always a good idea when dealing with value objects, as is the case here.

Let's compute the timestamps of the start and end of week when we build the `FormattedWeek` instance and keep them as fields. From the time included in the week, we get the start of the day (same date, midnight), and then subtract the index of the day in the week (Sunday is 0, Monday is 1, etc.). To subtract a day, we use the method `AddDate`. This method allows us to add and/or subtract years, months, and days to a `time.Time`. The method covers all use cases that would be painful to write manually, such as leap years or winter/summer time changes. It's good to keep in mind that adding a month is implemented as adding 31 days. As a result, adding 1 month to January 31, in ordinary years,

returns March 3 (31 days after January 31), which might be surprising. `AddDate` is very useful because it can be used to travel both in the future and in the past, as it allows a negative number of years, months, or days as its parameters:

```
start := startOfDay(include).AddDate(0, 0, -int(include.Weekday()))
```

From there, add seven days and remove 1 minute to get the end of the week. If we don't remove at least 1 ms, the end falls on Sunday morning next week, and formatting that date won't give us the proper day. We chose to remove a full minute to ease our job when debugging and testing; we could remove a full day as long as we don't need to display the time at all, only the date. This also covers the uncommon problem of leap seconds. To remove a minute, we'll use `Add`. Why are we using two different methods, `Add` and `AddDate`, to do the same operation? The answer is that their scope isn't really the same—`Add` should be used for durations shorter than a day, and `AddDate` should be used for anything beyond that. The following listing shows how to use the `NewFormattedWeek` function.

Listing 11.36 week.go: New formatted week

```
// FormattedWeek defines ...
type FormattedWeek struct {
    start, end time.Time #1
}

// NewFormattedWeek builds ...
func NewFormattedWeek(include time.Time, layout string) FormattedWeek {
    start := startOfDay(include).
        AddDate(0, 0, -int(include.Weekday())) #2
    return FormattedWeek{
        start: start,
        end:    start.AddDate(0, 0, 7).
            Add(-1 * time.Minute), #3
        layout: layout,
    }
}

// startOfDay returns the same date as given, at midnight.
func startOfDay(t time.Time) time.Time {
    year, month, day := t.Date()
    return time.Date(year, month, day, 0, 0, 0, 0, t.Location())
}
```

#1 Keeps the computed timestamps

#2 Computes the start of the week that includes the parameter

#3 Computes the end of the week

Typically, logic needs to be strongly unit tested, with test cases that go over months and over years; input times on Sundays, Mondays, and Saturdays; non-UTC time zones, during daylight saving time changes; and so on.

Finally, we need a way to format these two timestamps in a human-readable way. The `time` package has a function for this—it takes a formatting string as a parameter, for example, `"02 January 2006"`. It has to be in that format for Go to understand it.

We could hardcode this string as a constant of the package, but is it really the `habit` package's job to decide how things should be displayed? Ideally, we would pass that parameter to the `Start` and `End` methods. Because we first want to see how a function can be called via the templates without the complexity of a parameter, we decided that the week would be formatted (hence the name of our structure), and the

layout would be passed to `NewFormattedWeek` and kept as a field. It makes the methods' implementation quite straightforward.

Listing 11.37 Start and End methods

```
// Start returns the formatted first day of the week.
func (w FormattedWeek) Start() string {
    return w.start.Format(w.layout) #1
}

// End returns the formatted last day of the week.
func (w FormattedWeek) End() string {
    return w.end.Format(w.layout) #1
}
```

#1 Formats the time using the layout field

All that is left to do is to update `NewFormattedWeek` accordingly.

SIDE QUEST 11.1

Set Monday or Saturday as the first day of the week, and update the tests accordingly. You can find our solution (we chose Monday because we're from Europe) in the book's repository under a different build tag (<https://mng.bz/OBzR>).

MULTIPLE TEMPLATE ARGUMENTS

Back in the index page, build the week that you want to pass to the template execution:

```
week := habit.NewWeek(time.Now(), "02 January 2006")
```

The `Execute` method on the template can only take one parameter, and we already need to send the habits—how can we pass more values, such as this timestamp? One option is to create a clutter of a local struct, containing lists of habits and dates, but that can become wordy to use. A

simple `map` can do the job instead, as shown in the following listing. The template will receive the `map` as the dot and can access its values by using the keys as if there were fields.

Listing 11.38 `index.go`: Passing more data

```
err = tpl.Execute(w, map[string]interface{}{
    "Habits": habits, #1
    "Date":   week,
})
if err != nil {...}
```

#1 The template can access `.Habits`.

Add a line in the HTML to show the date. In Go, a function is similar to a field:

```
<p>This period covers {{.Date.Start}} -> {{.Date.End}}.</p>
```

You might notice that this line only deals with the `Date` field. If this were Go code, we could have a function accepting a `Date` and returning the start and end, and we would have factored this. While it's possible to declare variables in templates to help with readability, we don't recommend it. Their scope is too loose. Instead, we can restrict the template's visibility to only the `.Date` field using the `with` action:

```
{{with .Date}}<p>This period covers {{.Start}} -> {{.End}}.</p>{{end}}
```

This syntax sets the “local value” of the dot to `.Date` while inside the `{{ with }} ... {{ end }}`. Of course, we renamed the `slice` of habits from `.` to `.Habits` when we passed multiple arguments to our template, so this needs to be reflected in the `{{range}}` section. We can even wrap the whole block in a condition, so that when the `slice` is empty, the list isn't displayed at all.

Listing 11.39 index.go: Hiding an empty list

```
{{- if .Habits }} #1
<!-- titles and nice displays -->
<ul>
  {{- range .Habits }}
  <li>
    ...
  </li>
  {{- end }}
</ul>
{{- end }}
```

#1 Using the - will discard this empty line from the generated document.

Update your tests, and add a test case where the `slice` is empty. Figure 11.10 shows the result in the browser.

Habits Viewer

Objective: Weekly frequency:

This period covers 21 April 2024 -> 27 April 2024.

Here are your registered habits, you have 2.

- Call your mom 0 out of 1 [Tick!](#)
- Sleep 3 out of 7 [Tick!](#)

Figure 11.10 Browser result with the week details

Do you see why the test can't possibly be stable in the current situation? We're still using `time.Now()` as the input for our week. Next week, the test will be broken.

SIDE QUEST 11.2

Now that we know the status of our habits for the current date, it would be nice to check the previous weeks and anticipate the next ones. Try it at home by following these steps:

1. Add links for navigation in the HTML, and implement `Previous` and `Next` on the formatted week:

```
<a href="/?week={{ .Date.Previous }}">Prev.</a>
```

2. Pass the time as a query parameter to the index, defaulting to `time.Now()` if it's absent. Remember, the difference between path and query parameters is that the former is mandatory, whereas the latter is not. Use `r.URL.Query().Get("week")` to retrieve a value from the URL query.
3. Update the client call to include a timestamp of the habits we want to retrieve.
4. Add a query parameter to the unit test request, and update the output.
5. Should it be possible to tick something in the past? You decide. How should that be shown in the UI? Does the redirection after a tick still work, or does it go back to the current week? How can you fix this?
6. Should it be possible to navigate to the future? Does it make sense? How can you block the navigation to future weeks? Hiding the `Next` button isn't enough: What if the user passes a future timestamp as a query parameter?

11.5.2 Calling functions

As we've already seen, the template engine can access values, but it can do a lot more than that. In fact, we can run functions from the templates. From a certain angle, some actions, such as `range` and `if`, could be considered as functions in `{{ range . }}` and `{{ if . }}`, and we've already met `{{ len . }}`. In Go, you can write your own functions and call them in the template.

REGISTERING NEW FUNCTIONS IN THE TEMPLATE

Currently, we only show the number of times a habit was ticked. But this isn't fun enough. A web page should be entertaining, full of colors, and enticing its visitor to return. Let's add something along these lines here.

For instance, we could display a different color for each habit, reflecting how much of the weekly objective has been completed so far. Let's write a function that computes the progression and returns a label for different thresholds, as shown in listing 11.40. We can then use this label as a CSS class—we'll use a `string` for the label. This will replace the previous `done` class.

Listing 11.40 `index.go`: Label for progression

```
func statusCSSClass(habit *habit.Habit) string {
    prog := float32(habit.Ticks) /
        float32(habit.WeeklyFrequency) #1
    switch {
    case prog == 0:
        return "not_started"
    case prog < 0.5:
        return "started"
    case prog < 1:
        return "good_progress"
    default: #2
        return "completed"
    }
}
```

#1 Computes the progression

#2 One or more if there are more ticks than the target

To call this function from the template, the function needs to be registered in the template by calling the `Funcs` method when we create the template, as shown in listing 11.41. The `Funcs` method accepts a map of functions, where the keys are the names of these functions as they'll appear in the template `{{ myFunc . }}` (and the values are the Go functions to be called).

Listing 11.41 `index.xhtml`: Providing the function to the template

```
tpl, err := template.New("index").
    Funcs(template.FuncMap{
        "statusCSSClass": statusCSSClass, #1
    }).
    Parse(indexPage)
if err != nil {...}
```

#1 Maps the template's function name to a Go function

In the template, you can now use the identifier of the function, and give it the habit as an argument. Inside the `{{ range . }}` loop, the current habit is identified by a dot, which means we can retrieve the CSS class as shown in the following listing.

Listing 11.42 index.xhtml: Using a function

```
<style> #1
  li.habit.completed {
    background: #afa;
  }
  li.habit.good_progress {
    background: #ffa;
  }
  ...
</style>
...
<ul>
  {{- range .Habits }}
  <li class="habit {{ statusCSSClass . }}"> #2
    <b>{{ .Name }}</b>
```

#1 Declares the styles in the <header> section

#2 Calls the function

Run it. What do you think? Feel free to change the colors, of course. Don't forget to complete the CSS to support all the different labels. Write a test for the function, and then update the test for `index`.

PIPELINES

Templates in Go enable you to use pipelines. They should be used when the output of a function has to be used as the input of the next. Unix users will recognize the syntax as it's exactly the same: `{{ func1 . | func2 }}` produces the same result as calling `func2` on the output of `func1(.)`, which we usually write as `func2(func1(.))`. There's no limit to the number of `|` used in a pipeline.

Let's consider an example outside the scope of this chapter: suppose we need to print the hostnames that appear in a list of URLs. We also want to normalize this output—we want all hostnames to be lowercase. First, we'll need a function (in Golang) to extract the hostname from the URL. We can do this with the `net/url` package.

Listing 11.43 Pipelines: Extracting the domain name from a URL

```
func extractDomainName(rawURL string) string {
    u, err := url.Parse(rawURL)
    if err != nil {
        return "" #1
    }

    return u.Hostname()
}
```

#1 Errors sent to the template won't really be used.

Now, we can register this function in the template we would use for this side project. Because we'll also need to lowercase the whole string, let's also register that. But why not call `strings.ToLower` directly in `extractDomainNames` instead of dealing with the hassle of registering two functions? The following listing shows how we would build the template.

Listing 11.44 Registering all the functions using the pipeline

```
tmpl, err := template.New("domains").
    Funcs(template.FuncMap{
        "lower": strings.ToLower,
        "extractDomainNames": extractDomainNames}).
    Parse("{{range .}}{{extractDomainNames . | lower | println}}{{end }}")
```

Here, we added an extra `println` to have every hostname printed on its own line (the default action is to print with no empty line). Calling this on the `slice`

```
[]string{"http://google.com", "https://www.wikipedia.fr", "http://UN.org"}
```

produces the following:

```
google.com
wikipedia.fr
un.org
```

When it comes to pipelines, it's equivalent to write `{{ . | len }}` and `{{ len . }}` as both will print the length of the value. The difference here is to put some emphasis on the value on which we operate, rather than on the function we use. Both syntaxes are valid and lead to the same result. As

the developer, you'll decide which one corresponds to your situation.

11.5.3 Using define to declare functions

Go's templates offer the last action we'll be looking at in this chapter: writing functions as template actions instead of providing them from Go via `Funcs()`. One of the implications of this feature is that if a program is in charge of loading the template and executing it, we could update the behavior of the program by simply updating the template file it loads. This is dangerous on many levels (you need to keep track of the template currently being used, you need to make sure the template is always correct, etc.), but sometimes it's an interesting feature to have.

Let's take a look at our HTML template: we currently iterate through the habits with `{{- range .Habits }}`, and there are five lines before we reach the associated `{{- end }}` statement. In these five lines, the only thing we achieve is to turn a structure into a pretty list item. This is something a function could do, which would clarify the code a bit.

Here, we'll define a new template whose role is to format a habit into its HTML representation. Let's start by defining a new template in an HTML file. We could define it in the same file as our whole page, but we'd have to place it somewhere where it wouldn't make the HTML skeleton scream. Instead, we'll simply write it to another file, as shown in listing 11.45. Actually, because we're starting to have more than one file for our template, let's move our `index.xhtml` to a new folder, `internal/handlers/templates`. That's where we can create the new template file in charge of converting the habit to an HTML item: `habititem.xhtml`.

The syntax to define a template is relatively short: we use the `{{ define "templateName" }} ... {{ end }}` action. We'll simply move the contents of the `{{ range }}` loop from `index.xhtml` into this file.

Listing 11.45 `habititem.xhtml`: Declaring a template in another file

```
{{- define "habitItem"}}
    <li class="habit {{ statusCSSClass . }}">
        <b>{{.Name}}</b>
        <span class="{{ if ge .Ticks .WeeklyFrequency -}} done
-{{- end -}} "> {{- progress . }}</span>
        <a href="/tick/{{ .ID }}">Tick!</a>
    </li>
{{- end }}
```

At this point, a few notes on formatting are required. As you can see, both `{{- define }}` and `{{- end }}` start with the `-` to ignore previous spaces, with the `\n` preceding them. But we could also instead remove the `\n` after them, and use `{{ define - }}` and `{{ end - }}` (or even use the `-` on each side). Unfortunately, there's a side effect to that: `-` removes all subsequent space characters, including any indentation. While this wouldn't break the generated HTML document's rendering, it would make its source code look broken. And this is the other important point of this file: its contents must be properly indented. Go doesn't provide a function in its templates for this, but Helm—a tool that heavily relies on Go templates to work with YAML files—does, and we frequently see an action end with `| indent 4 }}` in Helm files.

We now have to update the `index.xhtml` file to call the new template: simply replace the section we copied with the action `{{ template "habitItem" . }}`. This will call the other template with the value of the dot, which, at that time, is iterating through the habits. Sometimes, the call to another template via `{{ template "habitItem" }}` will omit the dot—it's the default parameter. We decided to keep it:

```
{{- range .Habits }}  
    {{- template "habitItem" . }}  
{{- end }}
```

Again, we decided to indent the action for clarity—but you’ll notice that the indentation before the call to “habitItem” will be discarded by the - in the {{- define piece of code.

Now that we’ve moved our file to a new folder, however, we need to adapt how we load it. We didn’t write anything like `import habititem.xhtml` in the `index.xhtml` file, so how can it refer to a template in a different file? We used to embed the template file, but now, should we embed several files? What would a good limit be before we end up embedding a nonsensically long list of files? We don’t want to decide for you, but we think that anything beyond one is already a lot, and, for that matter, we won’t embed each file. Instead, because we’ve placed both our template files into a single directory, we can embed all the contents of that directory! This will load them in memory at compilation time. Let’s look at the changes in `index.go` that this brings.

Listing 11.46 index.xhtml: Embedding the contents of a directory

```
//go:embed templates/*.xhtml #1  
var templates embed.FS  
  
...  
tpl, err := template.New("index").Funcs(template.FuncMap{  
    "statusCSSClass": statusCSSClass,  
    "progress":      progress,  
}).ParseFS(templates, "templates/*.xhtml") #2
```

#1 Indexes all the files in the templates directory with an .xhtml extension

#2 Parses all the files with an .xhtml extension in the templates directory

It’s important to limit the scope of which files are embedded. There’s no point in preloading an image or any file that doesn’t pertain to the rendering of the index page here. If we try running our program, we now face an error:

Error in index: template: "index" is an incomplete or empty template

We registered two files in our template engine. Which one should be used when we execute the template? We need to be explicit, and ask for a specific template to be run.

Because we've loaded files, the only way to point at one template in particular is to use the filename. The `ParseFS` function already registers the files by their base name. Instead of calling `Execute`, we'll call `ExecuteTemplate`, which takes the extra argument of the base name of the template file we want to execute:

```
err = tpl.ExecuteTemplate(w, "index.xhtml", map[string]interface{}{
    "Habits": habits,
    "Date":   week,
})
```

Note We mentioned earlier that `New` registers templates using the name provided, when we use `Parse`. For this reason, we could have called `tpl.ExecuteTemplate(w, "index")` from the beginning with the same effect. But when there's only one template available, and it's been loaded with `Parse`, there's no ambiguity, and Go can determine what to do on its own.

Oh, in case you wonder—yes, it's possible to have a template's definition call itself. Please, just don't, for everyone's sake. If you really need recursion, try and do it in a piece of code, generating data ready to be displayed rather than computing it on the fly while displaying it.

That's it! We were able to define the equivalent of a function, which allowed us to factor our HTML document template and isolate the logic relative to a habit. If we had opened the template files with `os.Open` instead of using `//go:embed`, we would access whatever contents are present in the files every time the endpoint is called, which could lead to a lot of malicious scenarios.

Let's imagine another feature: we want users to be able to pick their favorite color for the interface. Somehow, we've stored that the current user likes dark mode after 10 P.M., and wants a light blue that they picked for daylight. We could simply give a class to the body and arrange the CSS so that colors correspond to either day or night. But if each user has their own preferences, it becomes interesting to use a template to generate the CSS on the fly. In a real production project, overriding in the HTML is probably better than generating the whole thing, but let's have fun.

For this, we'll create a new file, which will contain a new template. We can first try with just one variable.

Listing 11.47 `styles.css`: A text template

```
html {  
    background-color: {{.Background}}; #1  
    color: {{.Foreground}};  
}
```

#1 Passes the background color

The corresponding handler would look like what we had at the start of the chapter, except for a few details and one major change: we're not creating HTML anymore, so we need to use the more basic text/template package. The main difference is that the first escapes a lot of different characters, and the second does not. We don't want our background color to be hijacked by the dangerous value `</html>`.

Additionally, to tell the browser that this piece of text is to be used as a style sheet, we need to explicitly state the content type. Let's write this new file in the `internal/handlers/templates` directory, alongside the other `.xhtml` template files.

Listing 11.48 `styles.go`: Applies the CSS

```
import (
    "embed"
    "text/template" #1
    ...
)

//go:embed templates/styles.css
var stylesPage string

func (s *Server) styles(w http.ResponseWriter, r *http.Request) {
    tpl, err := template.New("styles").Parse(stylesPage)
    if err != nil {...}

    w.Header().Add("Content-Type", "text/css") #2

    err = tpl.Execute(w, map[string]interface{}{
        "Background": "DarkSlateGray",
        "Foreground": "LightGray"}) #3
    if err != nil {...}
}
```

#1 Uses text templates

#2 Explicitly states the content type

#3 Applies colors

Expose the endpoint, link the CSS inside the `<head>` tag of your index, and run it. Don't forget to test it as well.

You can go further by replacing the color string in the preceding code with a full palette that you pull from a database. The path to your main stylesheet becomes `styles.css?mode=dark`, for example. Or the color scheme can be linked to the user's preferences, the time zone (don't do this, VPNs make the user's time zone unreliable), or how far in the past we're browsing. You can even generate random colors and border styles. This is when giving a free rein to backend developers to pick designs becomes unmanageable..

Of course you can also generate deployment files in the same way, or, why not, Go code as well—Go code that generates Go code that generates . . . you get the picture.

Summary

- Golden files are used in tests to compare the expected contents of a call to the real output.
- Use `go mod edit -replace=old[@version]=new[@version]` to replace or override where Go fetches the module. This is common when using branches of libraries or when you want to use a local version on your computer.
- Templates are used to replace `fmt.Sprintf` or `strings.Builder`. They should be used when the blob of text that needs to be printed would make the two other options very cumbersome.
- Templates can be created using `text/template` or `html/template`. The main difference is that the HTML implementation protects the values printed against any HTML injection.
- It's best to write templates in different files than source code and to load their contents using `embed`.
- The `//go:embed` directive can be used to access (read-only) files as an `embed.FS`, a filesystem. This is quite useful when you need to load everything inside a directory.
- Load a template with `New()`, and provide a name, followed by `Parse`.
- Templates are based on actions, which appear between curly braces `{{ ... }}`.
- Templates use the current variable, called the dot, and represented by a period.
- `{{ . }}` is equivalent to Go's `fmt.Sprintf("%v", var)`.
- `{{.}}` and `{{ . }}` are equivalent.
- `{{- . }}` removes the space before the curly braces, while `{{ . -}}` removes the space after them.

- Templates and the codebase are heavily coupled—the template string needs to know the names of fields and methods that structures expose.
- Templates can access exposed fields of structures with `{{ .Field }}`.
- Templates can access methods, but we don't recommend it with `{{ .Method }}`. Only methods that return a printable value (with or without an error) can be called from a template.
- Unexposed fields and unexposed methods can't be accessed by templates.
- To iterate through a slice, array, map, or channel, use `{{range $index, $value := .}} ... {{end}}`.
- Alternatively, use `{{ range . }} ... {{end}}`; inside that range, the value of `.` will iterate through the values of the slice, array, channel, and map (values), not the index/keys.
- Use `{{ if <cond> }} ... {{ else if <cond> }} ... {{ else }} ... {{end}}` to express conditional behaviors. You can use Boolean operators (and, or, etc.) and numeric comparisons (lt, lte, gt, gte, ne, eq).
- Use `{{with .Date}} <section where we use only .Date subfields> {{end}}` to limit the length of accessors used: `.Date.Start` becomes `.Start` here.
- It's possible to register your own functions and make them available in the template: use the `Funcs` method when creating it.
- Pipelines can be used in templates to sequence actions. The pipeline character is the pipe `|`. Using a pipeline takes the output of the action on its left for the input of the action on its right.
- `{{ define "name" }} ... {{ end }}` can be used to declare the equivalent of functions. To call these functions, use

```
{{ template "name" . }}.
```

- When several templates are available, be explicit about which one should be executed. Templates that were loaded from a filesystem are indexed on the base name of the files that were loaded. When several template definitions are loaded, use `ExecuteTemplate` instead of simply `Execute`, and provide the name of the root template.

12 Go for other architectures

This chapter covers

- Exploring WebAssembly and TinyGo
- Developing and integrating Go, JavaScript, and HTML via WebAssembly
- Compiling, testing, and debugging WebAssembly applications
- Investigating TinyGo's unique features
- Flashing a microcontroller using TinyGo

Congratulations on the progress you've made and the knowledge you've gained throughout this journey. We hope you found it fulfilling! Consider this chapter an added bonus where we explore two technologies revolving around Go: WebAssembly (Wasm) and TinyGo. Both are ways of writing and compiling code for small processors, such as those found in web browsers or microcontrollers used in robotics and the Internet of Things (IoT)—environments where memory and CPU resources are limited.

Your projects so far have been running on your machine, but most of the time, we have to write programs that will end up running on machines other than our own. Those machines might be different from ours regarding available resources, operating system, CPU architecture, or the networks they're connected to. Go can help us build executables for specific environments. As an example, we'll start by exploring Wasm.